

Методичка по курсу «Распределенные вычисления»

Ф.С. Пеплин

26 ноября 2024 г.

Оглавление

1	Лекции	5
1.1	Введение в курс. Классификация параллельных систем . . .	6
1.2	Архитектура компьютера. Параллельное железо	10
1.3	Оценка производительности параллельных программ. По- лучение и отправка сообщений в MPI	17
1.4	Коллективные коммуникации в MPI-программах	22
1.5	Коллективные коммуникации (продолжение). Производные типы данных	30
1.6	Параллельная сортировка в MPI. Дэдлоки.	36
1.7	OpenMP	37
1.8	OpenMP	38
1.9	OpenMP	39
1.10	CUDA: создание гетерогенных программ	40
1.11	CUDA: варп шафл и общая память	48
1.12	CUDA: синхронизация потоков, банки памяти	49
2	Семинары	51
2.1	Pthreads: создание и завершение потоков	52
2.2	Pthreads: мьютексы и семафоры	59
2.3	Pthreads: потокобезопасность	71
2.4	Pthreads: барьеры и условные переменные	75
2.5	MPI: отправка и получение сообщений	81
2.6	MPI: практика коллективных коммуникаций	84
2.7	MPI: Scatterv, Gatherv	90
2.8	MPI: производные типы данных	98
2.9	OpenMP: потоки, таскинг	105
2.10	OpenMP: очередь	106
2.11	OpenMP: вычислительная геометрия	107
2.12	CUDA	108
2.13	CUDA	109
3	Лабораторные работы	111

<i>ОГЛАВЛЕНИЕ</i>	3
3.1 Pthreads	112
3.2 MPI	115

Глава 1

Лекции

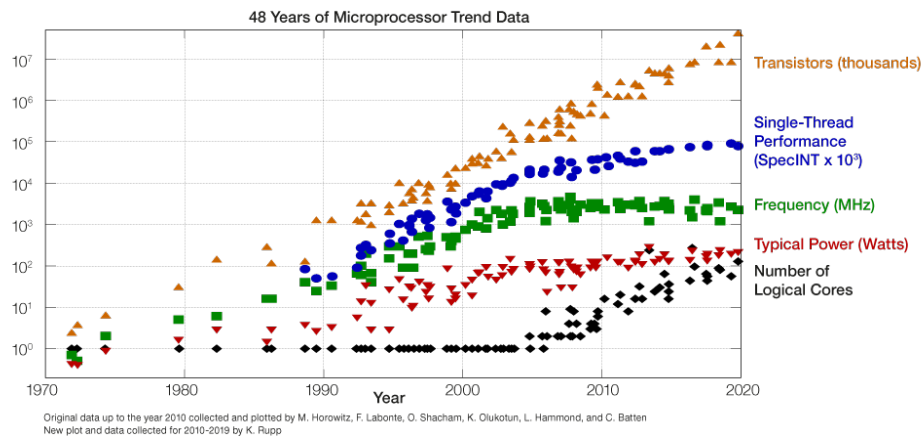


Рис. 1.1: Иллюстрация закона Мура

1.1 Введение в курс. Классификация параллельных систем

Формула оценивания

$$G = 0.5 \times \text{Exam} + 0.5 \times \text{Labs}$$

Закон Мура (1965): каждые два года количество транзисторов в интегральных схемах удваивается. Из рис. 1.1 видно, что общее количество транзисторов действительно увеличивается экспоненциально, однако производительность одноядерных процессоров, которая росла на 50% в год в период 1986 — 2003, в настоящее время стагнирует. Таким образом, рост производительности осуществляется за счет размещения нескольких самостоятельных ядер на одном процессоре.

Почему нельзя производить более мощные одноядерные системы?

- $\uparrow$ скорость $\Rightarrow \uparrow$ мощность $\Rightarrow \uparrow$ $^{\circ}\text{C}$ (при выполнении закона Мура в 2010 году тепловыделение было бы как в атомном реакторе)
- Скорость доступа к памяти не может быть выше скорости света $\Rightarrow$ при частоте 1 ТГц и памяти 8 Тб на 1 байт будет приходиться площадь атома

Решения проблемы

1. Квантовые вычисления
2. Развитие алгоритмов

1.1. ВВЕДЕНИЕ В КУРС. КЛАССИФИКАЦИЯ ПАРАЛЛЕЛЬНЫХ СИСТЕМ

3. Использовать многоядерные процессоры

Сферы применения НРС

1. Биотехнологии, биомедицина
 - Фолдинг белка
 - Персонализированная медицина, моделирование действия препаратов
2. Анализ данных
3. Вычислительное материаловедение
4. Расчеты в промышленности
 - Прочность, жесткость, устойчивость
 - Гидроаэродинамика, газовая динамика
 - Тепломассоперенос
5. Моделирование климата
6. Системы реального времени

Классификация параллельных систем

1. По используемой памяти
 - Общая память (Shared memory, рис. 1.2)
 - Распределенная память (Distributed memory, рис. 1.3)
2. По потокам данных и инструкций (таксономия Флинна)
 - Single Instruction — Multiple Data (SIMD) (параллелизм данных): общее устройство управления, разные операционные автоматы
 - Multiple Instruction — Multiple Data (MIMD) (параллелизм задач): разные устройства управления, разные операционные автоматы

Программы для систем с общей памятью называются многопоточными: в них один процесс распадается на несколько потоков. Программы для систем с распределенной памятью состоят из нескольких процессов.

Процесс

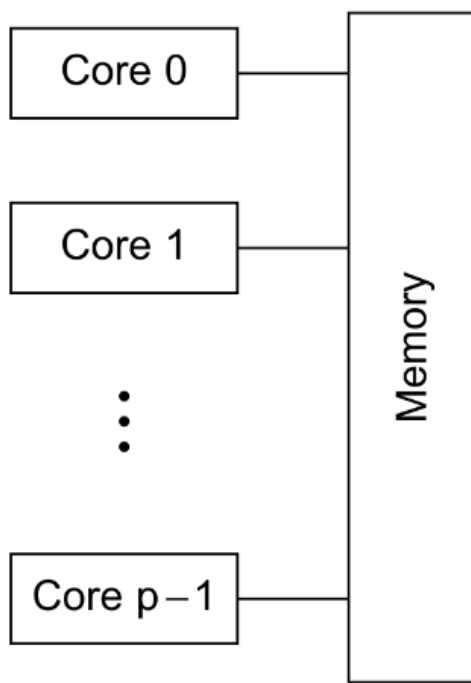


Рис. 1.2: Общая память

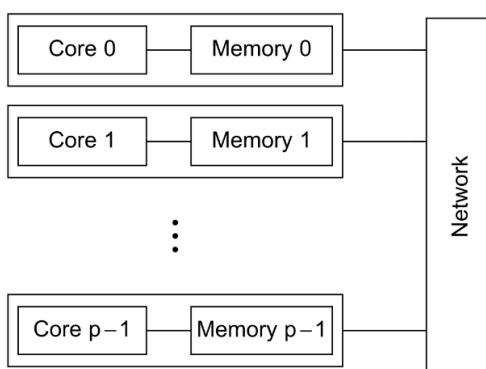


Рис. 1.3: Распределенная память

1.1. ВВЕДЕНИЕ В КУРС. КЛАССИФИКАЦИЯ ПАРАЛЛЕЛЬНЫХ СИСТЕМ

- Исполняемый код
- Дескрипторы ресурсов
- Стэк вызовов и куча
- Информация о статусе процесса (запущен, ожидает)
- Информация о правах доступа

Поток

- Тот же исполняемый файл, что и у процесса
- Те же ресурсы, кроме счетчика команд и стека вызовов

Одновременные вычисления (Concurrent computing) Различные задачи могут быть запущены одновременно.

Параллельные вычисления (Parallel computing) Различные процессы работают одновременно на разных вычислительных устройствах и взаимодействуют друг с другом для решения задачи.

Распределенные вычисления (Distributed computing) Программы взаимодействуют с другими программами для решения задачи.

Таким образом, параллельные и распределенные вычисления являются частными случаями одновременных вычислений. К одновременным вычислениям также можно отнести работу многозадачной ОС, даже на одноядерной системе.

1.2 Архитектура компьютера. Параллельное железо

1.2.1 Архитектура фон Неймана

В рамках данной модели компьютер представляется как центральный процессор, соединенный шиной с оперативной памятью.

В памяти единообразно хранятся данные и команды.

Центральный процессор состоит из двух частей: устройства управления (control unit) и операционного автомата (datapath). Устройство управления отвечает за определение инструкций, которые нужно выполнять. Операционный автомат занимается выполнением инструкций.

Процессор содержит регистры, в которых хранятся данные, с которыми в данный момент работает процессор. Один из регистров — счетчик команд, который содержит адрес следующей выполняемой процессором инструкции.

von Newmann bottleneck: Процессор выполняет инструкции в ≈ 100 раз быстрее, чем может получить доступ к памяти.

1.2.2 Улучшения архитектуры фон Неймана

Кэш-память

Кэш-строка. Объем данных, копируемый в кэш из оперативной памяти за одну операцию чтения. Конкретное содержание кэш-строки, формирующейся при доступе к ячейке памяти, определяется принципом пространственной и временной локальности. Размер ≈ 64 байт.

Попадание в кэш (cache hit). Требуемые процессору инструкции или данные находятся в кэше.

Кэш-промах (cache miss). Требуемые процессору инструкции или данные отсутствуют в кэше.

Неконсистентность кэша (cache inconsistency). Состояние, когда данные в основной памяти и в кэше отличаются.

Механизмы обеспечения когерентности кэшей:

Write-through — сквозная запись.

Write-back — обратная запись.

Ассоциативность кэшей. При необходимости записать новую кэш-строку в уже заполненный кэш, возникает вопрос: какую строку кэша удалить?

Кэш прямого отображения (direct mapped cache). Кэш-строка помещается в единственно возможное место в кэше (например, если кэш

1.2. АРХИТЕКТУРА КОМПЬЮТЕРА. ПАРАЛЛЕЛЬНОЕ ЖЕЛЕЗО11

состоит из четырех строк, то можно брать остаток от деления номера кэш-строки на 4 и записывать в соответствующее место в кэше).

Полностью ассоциативный кэш (fully associative cache). Кэш-строка может быть помещена в любое место в кэш-памяти.

n -ассоциативный кэш. Кэш-строка может быть помещена в одну из n локаций.

Если кэш полностью ассоциативный или n -ассоциативный, то для решения вопроса о том, в какое место записать новую строку, используется принцип LRU (Least Recently Used).

Виртуальная память

Единое адресное пространство для оперативной памяти и файла подкачки (свопа).

В целом та же идея, что и с кэшем, но есть два отличия:

1. Используется только write-through подход (по причине медленного доступа к жесткому диску)
2. Виртуальная память управляется комбинацией железа и ОС (кэш управляется только аппаратно).

Аналог кэш-строки — страница виртуальной памяти размером 4 — 16 Кб.

Чтобы обеспечить работу многозадачной операционной системы и исключить обращение нескольких программ по одному физическому адресу, при компиляции программы оперируют виртуальными адресами, которые преобразуются в физические средствами ОС.

Таблица страниц. Структура данных, используемая для преобразования виртуального адреса в физический.

TLB (Translation Lookaside Buffer). Быстрая память, содержащая 16 — 512 строк таблицы страниц. Строки выбираются исходя из принципа локальности.

TLB hit, TLB miss.

page fault — обращение к странице, которой нет в основной памяти (только на диске)

Параллелизм на уровне инструкций

ILP (Instruction Level Parallelism). Бывает двух типов.

Конвейерная обработка данных (pipelining) Выполнение команды разбивается на этапы, на каждом из которых используются отдельные функциональные устройства.

Множественная выдача команд (multiple issue) Одновременное выполнение команд различными функциональными устройствами.

Если функциональные устройства назначаются компилятором, то **статическая** м.в.к. Если функциональные устройства назначаются в процессе работы программы, то **динамическая** м.в.к. (**суперскалярность**).

М.в.к. осуществляется с помощью **спекуляции**.

1.2.3 SIMD

Одно управляющее устройство — много операционных автоматов.

Векторные процессоры

- Скорость и простота использования
- Рациональное использование данных
- Ограниченная масштабируемость
- Плохо приспособлены для работы с более сложными структурами данных
- Плохо приспособлены для работы с векторами малого размера

GPU (см. рис. 1.4)

- Большое количество функциональных устройств (≈ 128) на одном ядре.
- Не чистый SIMD/MIMD: на каждом SM несколько варпов, каждый из которых работает как SIMD.
- Комбинация общей и распределенной памяти.
- Неэффективны для небольших задач.

1.2.4 MIMD

Общая память

Все процессоры имеют приватный кэш первого уровня (L1 cache). Кэши второго и третьего уровней могут быть как приватными, так и общими.

MIMD-системы с общей памятью можно разделить на UMA (Uniform Memory Access) и NUMA (Non-Uniform Memory Access).

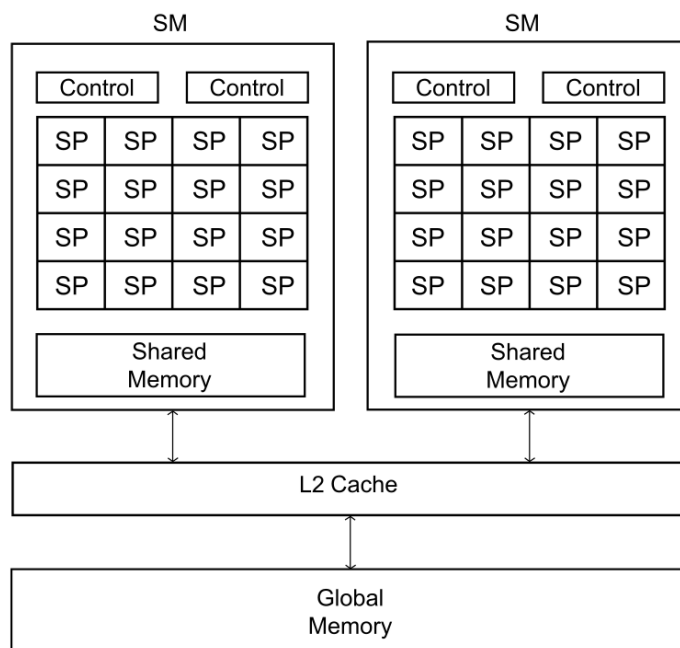


Рис. 1.4: Архитектура GPGPU

В случае UMA доступ ко всей памяти осуществляется напрямую: интерконнект соединяет все процессоры и все модули оперативной памяти. Поэтому время доступа к любой локации в памяти будет примерно одинаковым.

В случае NUMA интерконнект напрямую связывает процессор только со своей памятью. Доступ к «чужой» памяти осуществляется путем обращения к другому чипу, что занимает больше времени.

Распределенная память

MIMD-система с распределенной памятью — это кластер или грид-система.

Кластер. Состоит из отдельных узлов с общей памятью, соединенных сетью Ethernet.

Грид-система.

- Объединение сетей, находящихся друг от друга на большой расстоянии, в единую систему с распределенной памятью.
- В общем случае система гетерогенная.

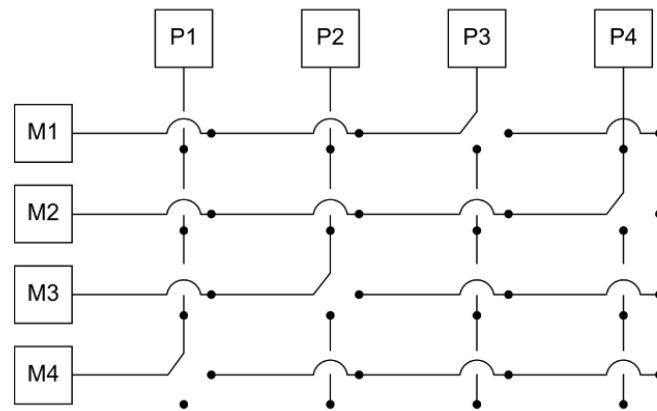


Рис. 1.5: Crossbar

1.2.5 Интерконнекты

Интерконнект — связь процессора с памятью.

Общая память

Для систем с общей памятью в качестве интерконнекта традиционно использовалась **шина** — параллельные провода, соединяющие устройства. Минус шины в том, что она не допускает подключение большого числа устройств. В настоящее время в системах с общей памятью используются переключатели-кроссбары (см. рис. 1.5), которые допускают одновременные коммуникации между сколь угодно большим количеством устройств. Однако стоимость такого интерконнекта слишком высока. В том числе по этой причине для большого количества процессоров чаще используется распределенная память.

Распределенная память

Ширина бисекции — два эквивалентных определения.

- Разбить систему на два множества с равным количеством узлов. Определить количество потоков данных, которые могут одновременно пересекать границу.
- Определить минимальное количество связей, которое нужно удалить, чтобы разбить систему на два несвязанных множества с одинаковым количеством узлов.

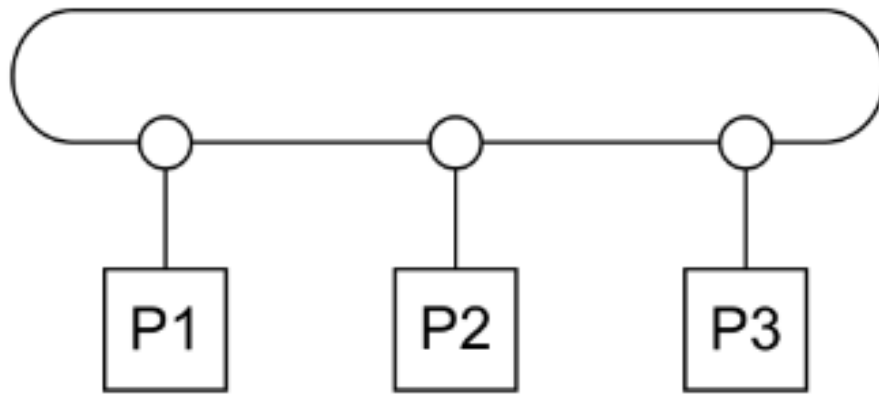


Рис. 1.6: Кольцо

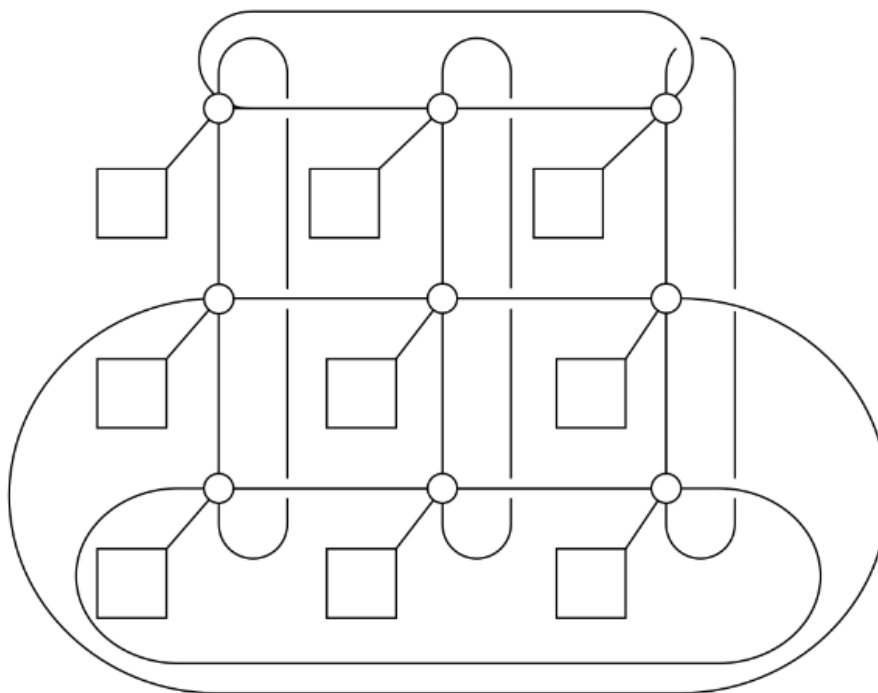


Рис. 1.7: Тороидальная сетка

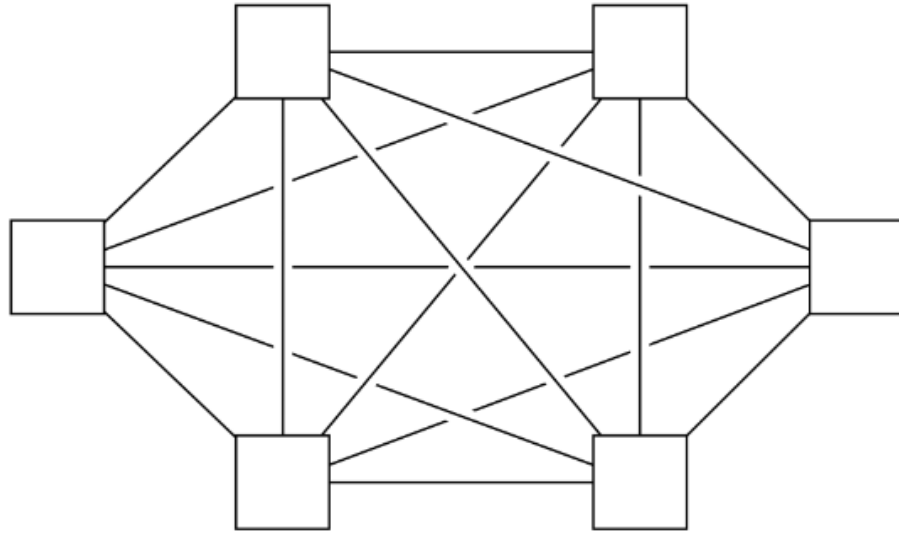


Рис. 1.8: Полностью связанный интерконнект

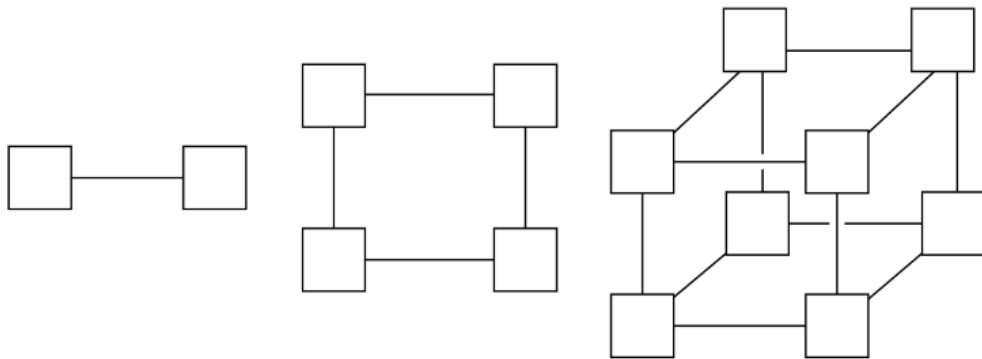


Рис. 1.9: Гиперкуб

Название	Ширина бисекции	Количество связей	Мощность адаптера
Кольцо	2	p	3
Тороидальная сетка	$2\sqrt{p}$	$2p$	5
Полносвязная сеть	$\frac{p^2}{4}$	$\frac{p^2}{2} - \frac{p}{2}$	p
Гиперкуб	$\frac{p}{2}$	$\frac{1}{2}p \log_2 p$	$1 + \log_2 p$

Таблица 1.1: Сравнение прямых интерконнектов

1.3 Оценка производительности параллельных программ. Получение и отправка сообщений в MPI

1.3.1 Оценка производительности параллельных программ

Ускорение параллельной программы S :

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}, \quad (1.1)$$

где T_{serial} — время работы программы без распараллеливания, T_{parallel} — время работы параллельной программы.

Эффективность параллельной программы E :

$$E = \frac{S}{p} = \frac{T_{\text{serial}}}{pT_{\text{parallel}}}, \quad (1.2)$$

где p — количество потоков/процессов.

Ясно, что

$$E \cdot T_{\text{parallel}} = \frac{T_{\text{serial}}}{p}. \quad (1.3)$$

Формула (1.3) подталкивает к интерпретации понятия эффективности как доли времени работы параллельной программы, которая тратится максимально эффективно. Иными словами, общее время работы программы можно представить как

$$T_{\text{parallel}} = \frac{T_{\text{serial}}}{p} + T_{\text{overhead}} \quad (1.4)$$

Источники возникновения T_{overhead} :

- Критические секции
- Синхронизация
- Коммуникация

Выведем оценку сверху для ускорения S :

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} \stackrel{(1.4)}{=} \frac{T_{\text{serial}}}{\frac{T_{\text{serial}}}{p} + T_{\text{overhead}}} < \frac{T_{\text{serial}}}{T_{\text{overhead}}} \quad (1.5)$$

Обозначим через r долю инструкций программы, которые не могут быть распараллелены. Предположим, что остальные $1 - r$ инструкций допускают распараллеливание без накладных расходов. Тогда $T_{\text{overhead}} = rT_{\text{serial}}$ и формула (1.5) примет вид

$$S < \frac{1}{r} \quad (1.6)$$

Формула (1.6) выражает собой **закон Амдала** (1967): ускорение выполнения программы за счет распараллеливания ее инструкций на множестве вычислителей ограничено временем, необходимым для выполнения ее последовательных инструкций.

Отметим, что при выводе закона Амдала мы зафиксировали количество инструкций и задались вопросом: «во сколько раз быстрее мы можем выполнить данное множество команд»? Полученная оценка является довольно пессимистичной. Другую, более радостную, зависимость, можно получить, если зафиксировать время выполнения последовательной программы и задать себе вопрос: «во сколько раз больше инструкций мы можем выполнить за то же самое время?». Чтобы ответить на этот вопрос, введем понятие *ускорения масштабирования* параллельной программы S_p :

$$S_p = (1 - r)p + r \quad (1.7)$$

Величина S_p может трактоваться как отношение количества инструкций, выполненных за единицу времени на параллельной архитектуре к количеству инструкций, выполненных за тот же промежуток времени программой без распараллеливания.

Из (1.7) следует, что объем работы, выполненный параллельной программой, почти в p раз больше, чем последовательной (если r мало). Объем инструкций, которые мы успеем выполнить на параллельной архитектуре, растет линейно при росте количества ядер. Данное утверждение является формулировкой **закона Густавсона** (1988).

1.3.2 Отправка и получение сообщений в системах с распределенной памятью

Листинг 1.1: Первая MPI-программа

```
1 #include <mpi.h>
2 #include <string.h>
3 #include <stdio.h>
4 #define MAX_SIZE 100
```

1.3. ОЦЕНКА ПРОИЗВОДИТЕЛЬНОСТИ ПАРАЛЛЕЛЬНЫХ ПРОГРАММ. ПОЛУЧЕНИЕ И

```
5
6 int main()
7 {
8
9     int comm_sz;
10    int my_rank;
11    char greeting[MAX_SIZE];
12
13    MPI_Init(NULL, NULL);
14
15    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
16    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
17
18    if (my_rank != 0)
19    {
20        sprintf(greeting ,
21            "Greetings_from_process_%d_of_%d", my_rank, comm_sz);
22        MPI_Send(greeting , strlen(greeting) + 1,
23            MPI_CHAR, 0, 0, MPI_COMM_WORLD);
24    }
25    else
26    {
27        printf("Greetings_from_process_%d_of_%d\n",
28            my_rank, comm_sz);
29        for (int i = 1; i < comm_sz; i++)
30        {
31            MPI_Recv(greeting , MAX_SIZE, MPI_CHAR,
32                i , 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
33            printf("%s\n", greeting);
34        }
35    }
36
37    MPI_Finalize();
38
39    return 0;
40 }
```

Все команды MPI должны помещаться между вызовами функций `MPI_Init` и `MPI_Finalize`. Первая из них принимает указатели на аргументы командной строки и выделяет место для буферов передаваемых и получаемых сообщений, решает, какой ранг будет у какого процесса и т.д.

Вторая функция освобождает выделенные ресурсы.

Коммуникатором называется множество процессов. Процессы могут отправлять и получать сообщения только в рамках одного коммуникатора. Коммуникатор `MPI_COMM_WORLD` состоит из всех процессов, которые запущены пользователем. В строках 15 и 16 происходит запись в переменные `comm_sz` и `my_rank` размера коммуникатора и порядкового номера текущего процесса в коммуникаторе соответственно.

Отправка и получение сообщений происходит путем вызова функций `MPI_Send` и `MPI_Recv`.

Сигнатура `MPI_Send`:

```
MPI_Send(void* buf,
         int size,
         MPI_Datatype,
         int dest,
         int tag,
         MPI_Comm communicator);
```

Первые три аргумента описывают передаваемое сообщение: где оно хранится, сколько элементов какого типа содержит. Четвертый аргумент — это номер процесса-получателя, пятый — тег, то есть условное обозначение, которое позволит получателю понять, какая информация содержится в этом сообщении. Последний аргумент — это имя коммуникатора.

Сигнатура `MPI_Recv`:

```
MPI_Recv(void* buf,
         int size,
         MPI_Datatype,
         int source,
         int tag,
         MPI_Comm communicator,
         MPI_Status* status_p);
```

Первые три аргумента характеризуют получаемое сообщение: в какой буфер его записать, каковы максимальный размер и тип элементов сообщения. Далее указывается ранг процесса-отправителя и тег сообщения, а также имя коммуникатора. Последний аргумент предназначен для вывода информации о ранге отправителя и теге сообщения. Зачем это нужно? Дело в том, что в качестве третьего аргумента функции `MPI_Recv` можно передать макрос `MPI_ANY_SOURCE`, а в качестве четвертого — `MPI_ANY_TAG`. Если указан хотя бы один из этих макросов, то мы можем не знать тега сообщения или его отправителя, или и того, и другого. Тогда нужно создать переменную типа `MPI_Status` и пере-

1.3. ОЦЕНКА ПРОИЗВОДИТЕЛЬНОСТИ ПАРАЛЛЕЛЬНЫХ ПРОГРАММ. ПОЛУЧЕНИЕ И

дать указатель на нее в функцию `MPI_Recv`. Функция `MPI_Recv` запишет в эту переменную информацию о теге и отправителе, которую можно получить, обратившись к полям `MPI_SOURCE` и `MPI_TAG`.

Что происходит, когда вызывается функция `MPI_Send`? В первую очередь передаваемая информация упаковывается в «конверт», который помимо содержательной части включает в себя служебные данные. Затем этот «конверт» может либо начать передаваться по сети получателю, когда тот вызовет соответствующую функцию `MPI_Recv`, либо происходит кэширование сообщения (сохранение во внутренней памяти) без его отправки. Обычно первый вариант развития событий происходит, когда размер передаваемого значения выше некоторого порогового значения, а второй вариант характерен для маленьких сообщений. Таким образом, когда программа возвращается из функции `MPI_Send`, мы не знаем точно, состоялась ли коммуникация с процессом-получателем.

Что же касается функции `MPI_Recv`, то возвращение из нее, напротив, говорит о том, что коммуникация состоялась.

Пусть процесс q вызывает функцию

```
MPI_Send(send_buf_p, send_buf_sz,
         send_type, dest, send_tag, send_comm);
```

Процесс r осуществляет вызов

```
MPI_Recv(recv_buf_p, recv_buf_sz,
         recv_type, source, recv_tag, recv_comm, &status);
```

Для того, чтобы случился акт коммуникации между процессами, нужно, чтобы

- `recv_comm=send_comm`
- `recv_tag=send_tag`
- `dest=r, source=q`

Для того, чтобы сообщение было передано успешно, нужно, чтобы дополнительно выполнялись еще условия

- `recv_type=send_type`
- `recv_buff_sz ≥ send_buff_sz`

1.4 Коллективные коммуникации в MPI-программах

Рассмотрим задачу нахождения значения определенного интеграла

$$\int_a^b f(x)dx \quad (1.8)$$

с использованием численных методов.

Согласно методу трапеций

$$\int_a^b f(x)dx \approx \frac{h}{2} (f(a) + f(b)) + h (f(x_1) + f(x_2) + \dots f(x_{n-1})), \quad (1.9)$$

где n — количество отрезков, на которые делится отрезок $[a, b]$,
 $x_0 = a, x_1 = x_0 + h, x_2 = x_0 + 2h, \dots, x_n = b$ — узлы разбиения,
 $h = \frac{b-a}{n}$ — шаг разбиения.

Первая попытка решить задачу приведена в листинге 1.2.

Листинг 1.2: Вычисление определенного интеграла

```

1 #include <mpi.h>
2 #include <string.h>
3 #include <stdio.h>
4
5 int n;
6 double a;
7 double b;
8
9 int comm_sz;
10 int my_rank;
11
12 double f(double x)
13 {
14     return x * x;
15 }
16
17 void Input()
18 {
19     if (my_rank == 0)
20     {
21         printf("Enter a, b, n\n");
22         scanf("%lf %lf %d", &a, &b, &n);
23         for (int i = 1; i < comm_sz; i++)
24         {

```

1.4. КОЛЛЕКТИВНЫЕ КОММУНИКАЦИИ В MPI-ПРОГРАММАХ 23

```
25     MPI_Send(&a, 1, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);
26     MPI_Send(&b, 1, MPI_DOUBLE, i, 0, MPI_COMM_WORLD);
27     MPI_Send(&n, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
28 }
29 }
30 else
31 {
32     MPI_Recv(&a, 1, MPI_DOUBLE, 0, 0,
33             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
34     MPI_Recv(&b, 1, MPI_DOUBLE, 0, 0,
35             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
36     MPI_Recv(&n, 1, MPI_INT, 0, 0,
37             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
38 }
39 }
40
41 double Trap(double a, double b, int n, double h)
42 {
43     double res = 0.0;
44     res = 0.5 * (f(a) + f(b));
45     for (int i = 1; i <= n - 1; i++)
46     {
47         double x = a + i * h;
48         res += f(x);
49     }
50     return res * h;
51 }
52
53 int main()
54 {
55
56     MPI_Init(NULL, NULL);
57
58     MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
59     MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
60
61     Input();
62
63     double h = (b - a) / n;
64
65     int local_n = n / comm_sz;
```

```

66  double local_a = a + my_rank * local_n * h;
67  double local_b = local_a + local_n * h;
68  double local_int = Trap(local_a, local_b, local_n, h);
69
70  double global_int;
71
72  if (my_rank != 0)
73  {
74      MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);
75  }
76  else
77  {
78      global_int = local_int;
79      for (int i = 1; i < comm_sz; i++)
80      {
81          MPI_Recv(&local_int, 1, MPI_DOUBLE, i, 0,
82                  MPI_COMM_WORLD, MPI_STATUS_IGNORE);
83          global_int += local_int;
84      }
85  }
86
87  if (my_rank == 0)
88      printf("Int = %lf\n", global_int);
89
90  MPI_Finalize();
91
92  return 0;
93 }

```

Основной недостаток программы 1.2 заключается в том, что нахождение глобальной суммы из локальных сумм целиком и полностью выполняется нулевым процессом. Выглядит более разумным использовать древовидную структуру (см. рис. 1.10). Это позволит снизить время с $O(p)$ до $O(\log(p))$ (p — количество процессов). Реализовать этот алгоритм можно с помощью `MPI_Reduce`:

```

MPI_Reduce(
    void*          input_buf_p,          /* in */
    void*          output_buf_p,         /* out */
    int            count,                 /* in */
    MPI_Datatype   datatype,             /* in */
    MPI_Op         operation,             /* in */

```

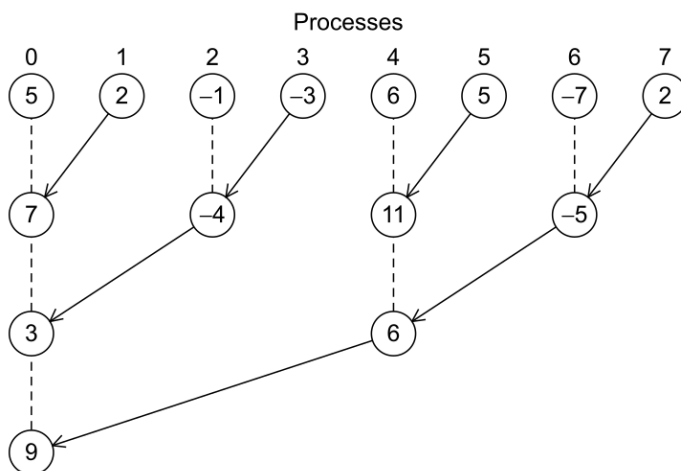


Рис. 1.10: Нахождение глобальной суммы с помощью MPI_Reduce

```

    int dest , /* in */
    MPI_Comm Communicator /* in */
);

```

Функция `MPI_Reduce` реализует **коллективную коммуникацию**. Эта функция берет из каждого процесса значения, на которые указывают `input_buf_p`, и суммирует их, так, чтобы результат оказался в буфере `output_buf_p` процесса `root`. Помимо суммирования, можно выполнять и другие операции, если передать соответствующие аргумент типа `MPI_Op`. Возможно вычисление произведения, нахождение минимума или максимума, логические и побитовые операции. Также можно определить свою собственную операцию.

Примеры других функций, реализующих коллективную коммуникацию

```

MPI_Allreduce(
    void* input_buf_p , /* in */
    void* output_buf_p , /* out */
    int count , /* in */
    MPI_Datatype datatype , /* in */
    MPI_Op operation , /* in */
    MPI_Comm Communicator /* in */
);

```

Работает также, как и `MPI_Reduce`, однако полученное значение затем передается всех процессам коммуникатора (см. рис. 1.11).

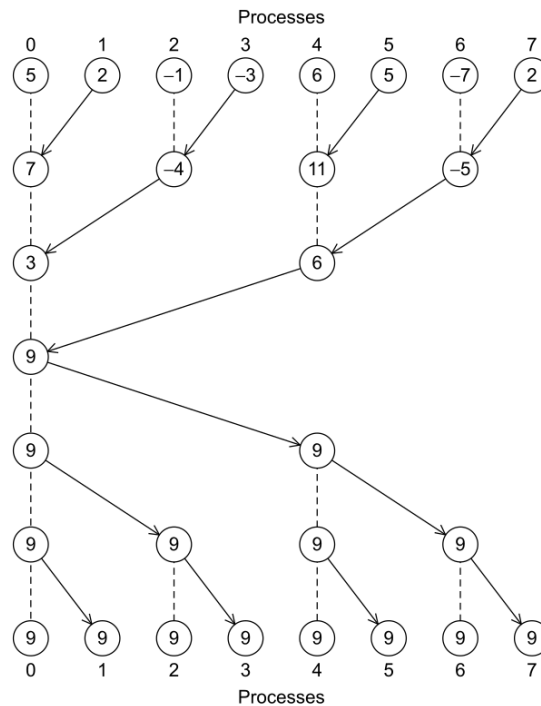


Рис. 1.11: Механизм работы MPI_Allreduce

```

MPI_Bcast(
    void*          input_buf_p,          /* in/out */
    int            count,                /* in */
    MPI_Datatype   datatype,            /* in */
    int            source_proc,          /* in */
    MPI_Comm       Communicator          /* in */
);

```

Данная функция берет значение, которое хранится в буфере `input_buf_p` на процессе `source_proc`, и отправляет его всем остальным процессам в коммунитаторе, записывая результат в переменную `input_buf_p`.

Работа функции `MPI_Allreduce` сводится к последовательному вызову `MPI_Reduce` и `MPI_Bcast`.

Перепишем программу из листинга 1.2, используя возможности коллективных коммуникаций. Результат см. в листинге 1.3

Листинг 1.3: Демонстрация силы коллективных коммуникаций

```

1 #include <mpi.h>
2 #include <string.h>
3 #include <stdio.h>

```

1.4. КОЛЛЕКТИВНЫЕ КОММУНИКАЦИИ В MPI-ПРОГРАММАХ 27

```
4
5 int n;
6 double a;
7 double b;
8
9 int comm_sz;
10 int my_rank;
11
12 double f(double x)
13 {
14     return x * x;
15 }
16
17 void Input()
18 {
19     if (my_rank == 0)
20     {
21         printf("Enter a, b, n\n");
22         scanf("%lf %lf %d", &a, &b, &n);
23     }
24     MPI_Bcast(&a, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
25     MPI_Bcast(&b, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
26     MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
27 }
28
29 double Trap(double a, double b, int n, double h)
30 {
31     double res = 0.0;
32     res = (f(a) + f(b)) / 2.0;
33     for (int i = 1; i <= n - 1; i++)
34     {
35         double x = a + i * h;
36         res += f(x);
37     }
38     return h * res;
39 }
40
41 int main()
42 {
43
44     MPI_Init(NULL, NULL);
```

```

45
46 MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
47 MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
48
49 Input();
50
51 int local_n = n / comm_sz;
52 double h = (b - a) / n;
53
54 double local_a = a + my_rank * local_n * h;
55 double local_b = local_a + local_n * h;
56 double local_int = Trap(local_a, local_b, local_n, h);
57 double global_int = 0.0;
58
59 MPI_Reduce(&local_int, &global_int, 1, MPI_DOUBLE,
60 MPI_SUM, 0, MPI_COMM_WORLD);
61
62 if (my_rank == 0)
63     printf("Int = %lf\n", global_int);
64
65 MPI_Finalize();
66
67 return 0;
68 }

```

Свойства функций, относящихся к классу коллективных коммуникаций.

1. Все процессы внутри коммуникатора должны вызывать эту функцию. Если хотя бы один процесс в коммуникаторе не вызовет функцию, то результат будет непредсказуемым.
2. Выходной буфер `output_buffer_p` используется только на процессе с рангом `root`. На других процессах можно передавать что угодно. Так, вызов в строке 59 листинга 1.3 можно заменить на

```

if (my_rank == 0)
    MPI_Reduce(&local_int, &global_int, 1, MPI_DOUBLE,
        MPI_SUM, 0, MPI_COMM_WORLD);
else
    MPI_Reduce(&local_int, &global_int, 1, MPI_DOUBLE,
        MPI_SUM, 0, MPI_COMM_WORLD);

```

3. Точечные коммуникации (`MPI_Send`, `MPI_Recv`) стыкуются на основе совпадения коммуникатора и тега. Коллективные коммуникации не используют тегов, поэтому их стыковка происходит на основе коммуникатора и порядка, в котором вызываются функции. Так, если запустить код из листинга 1.4 на трех процессах, то значения `b` и `d` будут 4 и 5, не 3 и 6.

Листинг 1.4: Стыковка коллективных коммуникаций

```

int a = 1;
int c = 2;
int b = 0;
int d = 0;

if (my_rank == 0 || my_rank == 2)
{
    MPI_Reduce(&a, &b, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&c, &d, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
}
else
{
    MPI_Reduce(&c, &d, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&a, &b, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
}

if (my_rank == 0)
printf("b=%d, d=%d\n", b, d);

```

1.5 Коллективные коммуникации (продолжение). Производные типы данных

1.5.1 Производные типы данных

При пересылке сообщений с помощью функции `MPI_Send` в качестве передаваемого типа можно только один из элементарных типов (целое или дробное число). Что же делать, если нужно передать сразу много значений разных типов? Т.к. коммуникация по сети обычно довольно медленная и требует дополнительных действий для каждого отправления, то более выгодно отправить одно большое сообщение, чем много маленьких. В этом случае полагается целесообразным создание нового типа.

Процедура создания нового типа для использования в функции `Input` программы 1.3 показана в листинге 1.5.

Листинг 1.5: Создание нового типа данных

```

1 void Build_mpi_type(double *a_p, double *b_p,
2   int *n_p, MPI_Datatype *mpi_derived_datatype)
3 {
4   int array_of_block_lengths[3] = {1, 1, 1};
5   MPI_Aint a_addr, b_addr, n_addr;
6   MPI_Get_address(a_p, &a_addr);
7   MPI_Get_address(b_p, &b_addr);
8   MPI_Get_address(n_p, &n_addr);
9   MPI_Aint array_of_displacements[3] = {0, b_addr - a_addr,
10    n_addr - a_addr};
11   MPI_Datatype array_of_types[3] = {MPI_DOUBLE,
12    MPI_DOUBLE, MPI_INT};
13   MPI_Type_create_struct(3, array_of_block_lengths,
14    array_of_displacements, array_of_types, mpi_derived_datatype);
15   MPI_Type_commit(mpi_derived_datatype);
16 }
```

Создание нового типа осуществляется с помощью функции `mpi_Type_create_struct`, которая принимает следующие аргументы:

1. Количество передаваемых элементов. Каждый элемент может быть либо одиночной переменной, либо массивом.
2. Массив длин, в котором указывается, сколько переменных в каждом элементе мы передаем. Т.к. мы планируем передавать три оди-

1.5. КОЛЛЕКТИВНЫЕ КОММУНИКАЦИИ (ПРОДОЛЖЕНИЕ). ПРОИЗВОДНЫЕ ТИПЫ ДАННЫХ

ночных переменных, то этот массив должен состоять из трех единиц. Если бы, например, второй элемент был бы массивом из 100 элементов, то второй компонент массива `array_of_block_lengths` был бы сотней.

3. Массив смещений адреса относительно первой передаваемой переменной.
4. Массив типов. Для каждого элемента нужно указать, какой у него тип.
5. Указатель на создаваемый тип, по которому он и будет записан.

Созданный тип регистрируется в MPI в 15 строке.

После использования типа нужно освободить выделенные для него ресурсы с помощью функции

```
MPI\_Type\_commit(  
    MPI\_datatype* mpi_dt    /* in/out */  
);
```

Использование пользовательских типов данных в функции `Input` листинга 1.3 показано в листинге 1.6:

Листинг 1.6: Использование пользовательских типов данных

```
void Input()  
{  
    MPI_Datatype abn;  
    Build_mpi_type(&a, &b, &n, &abn);  
    if (my_rank == 0)  
    {  
        printf("Enter a, b, n\n");  
        scanf("%lf %lf %d", &a, &b, &n);  
    }  
    MPI_Bcast(&a, 1, abn, 0, MPI_COMM_WORLD);  
}
```

1.5.2 Функции Gather, Scatter, Allgather

Напишем программу для суммирования двух векторов в предположении, что длины векторов делятся нацело на количество процессов. Для распределения массива между процессами и последующего сбора результатов для вывода будем использовать следующие функции коллективной коммуникации:

```

int MPI_Scatter(
    void*          send_buf_p,      /*in*/
    int            send_count,      /*in*/
    MPI_Datatype   datatype,       /*in*/
    void*          recv_buf_p,      /*out*/
    int            recv_count,      /*in*/
    MPI_Datatype   recv_type,      /*in*/
    int            src_proc,        /*in*/
    MPI_Comm       comm            /*in*/
);

```

Функция **Scatter** разделяет переданный ей массив на процессе **src_proc** на некоторое количество частей по **send_count = recv_count** элементов в каждом и отправляет их всем процессам в коммуникаторе.

```

int MPI_Gather(
    void*          send_buf_p,      /*in*/
    int            send_count,      /*in*/
    MPI_Datatype   datatype,       /*in*/
    void*          recv_buf_p,      /*out*/
    int            recv_count,      /*in*/
    MPI_Datatype   recv_type,      /*in*/
    int            dest_proc,       /*in*/
    MPI_Comm       comm            /*in*/
);

```

Функция **MPI_Gather** берет массивы **send_buf_p** с каждого процесса в коммуникаторе и собирает их в единый массив **recv_buf_p** на процессе **dest_proc**.

Также есть функция **MPI_Allgather**, сигнатура которой отличается от **MPI_Gather** лишь отсутствием **dest_proc**. Данная функция является последовательностью вызовов **MPI_Gather** и **MPI_Bcast**.

Листинг 1.7: Сложение векторов

```

#include <mpi.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

int comm_sz;
int my_rank;

int *c;

```

1.5. КОЛЛЕКТИВНЫЕ КОММУНИКАЦИИ (ПРОДОЛЖЕНИЕ). ПРОИЗВОДНЫЕ ТИПЫ Д

```
int n;
int* a;
int* b;
int *local_a;
int *local_b;
int *local_c;
int local_n;

void Input()
{
    if (my_rank == 0)
    {
        printf("Enter_n\n");
        scanf("%d", &n);
    }
    MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
    local_n = n / comm_sz;
    local_a = malloc(local_n * sizeof(int));
    local_b = malloc(local_n * sizeof(int));
    local_c = malloc(local_n * sizeof(int));
    if (my_rank == 0)
    {
        a = malloc(n * sizeof(int));
        b = malloc(n * sizeof(int));
        for (int i = 0; i < n; i++)
        {
            scanf("%d", &a[i]);
        }
        for (int i = 0; i < n; i++)
        {
            scanf("%d", &b[i]);
        }
        MPI_Scatter(a, local_n, MPI_INT, local_a, local_n,
            MPI_INT, 0, MPI_COMM_WORLD);
        MPI_Scatter(b, local_n, MPI_INT, local_b, local_n,
            MPI_INT, 0, MPI_COMM_WORLD);
        free(a);
        free(b);
    }
    else
```

```
{
    MPI_Scatter(a, local_n, MPI_INT, local_a, local_n,
               MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Scatter(b, local_n, MPI_INT, local_b, local_n,
               MPI_INT, 0, MPI_COMM_WORLD);
}

void add()
{
    for (int i = 0; i < local_n; i++)
        local_c[i] = local_a[i] + local_b[i];
}

void print()
{
    int *c = malloc(n * sizeof(int));
    MPI_Gather(local_c, local_n, MPI_INT, c, local_n,
               MPI_INT, 0, MPI_COMM_WORLD);
    if (my_rank == 0)
        for (int i = 0; i < n; i++)
            printf("%d_", c[i]);
        printf("\n");
    free(c);
}

int main()
{
    MPI_Init(NULL, NULL);

    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    Input();
    add();
    print();

    MPI_Finalize();

    return 0;
}
```

1.5. КОЛЛЕКТИВНЫЕ КОММУНИКАЦИИ (ПРОДОЛЖЕНИЕ). ПРОИЗВОДНЫЕ ТИПЫ Д

}

1.6 Параллельная сортировка в MPI. Дэдлоки.

Рассмотрим задачу параллельной сортировки массива методом четно-нечетной пузырьковой сортировки, который проиллюстрирован на примере.

1.7 OpenMP

1.8 OpenMP

1.9 OpenMP

1.10 CUDA: создание гетерогенных программ

1.10.1 Архитектура GPGPU

GPGPU (General-Purpose computing on Graphical Processing Unit) — использование графических процессоров для решения произвольных вычислительных задач (не только обработка графики).

CUDA — это API, используемое для программирования графических карт `nvidia`. Включает в себя библиотеку программного кода и модифицированный компилятор.

Графическая карта `nvidia` состоит из нескольких мультипроцессоров потоковой обработки данных (streaming multiprocessors, SM). Каждый SM содержит до 1024 процессоров (streaming processors, SP).

Каждый SP имеет приватную быструю регистровую память (типичное значение сотни байт). Каждый SM имеет общую (shared) память, на порядок медленнее регистровой, объемом в десятки килобайт. Есть также общая для всех SM глобальная память, еще на пару порядков медленнее, зато объемом в десятки гигабайт.

На каждом SP запускается один поток. Набор из 32 потоков называется варпом (warp). Потоки внутри одного варпа работают как одна SIMD машина, то есть синхронно выполняют одни и те же инструкции, каждый над своими данными.

Архитектура графического ускорителя схематически изображена на рис. 1.12.

Программирование графических ускорителей существенно отличается от программирования систем с общей или с распределенной памятью. Именно, при программировании графических процессоров мы вынуждены одновременно писать код как для хоста (центральный процессор и оперативная память компьютера), так и для девайса (собственно графический ускоритель). Программа начинается с выполнения хостового кода, из которого запускается код на графической карте. Последний называется ядром (kernel). См. рис. 1.13.

1.10.2 Гриды, блоки и функции

Блок — это совокупность потоков, запущенных на одной SM. Количество потоков в блоке содержится в переменной `blockDim`. Номер потока внутри блока находится в переменной `threadIdx`.

Грид — это набор блоков. Количество блоков находится в переменной `gridDim`. Номер блока в гриде находится в переменной `blockIdx`.

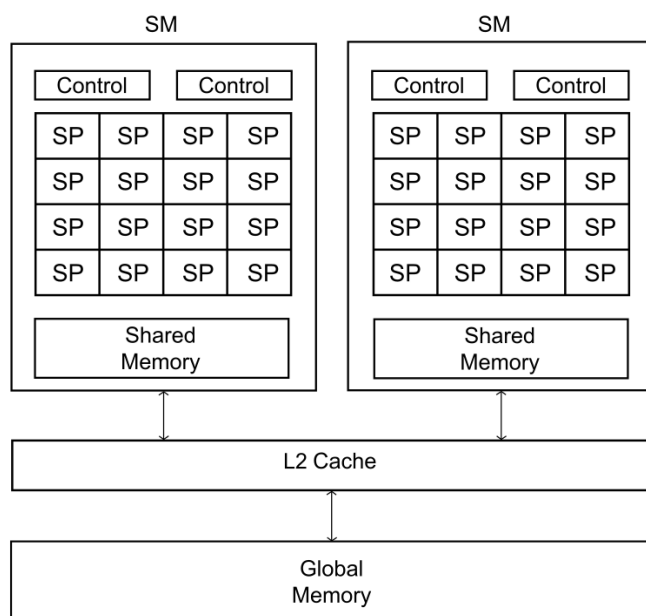


Рис. 1.12: Архитектура графической карты

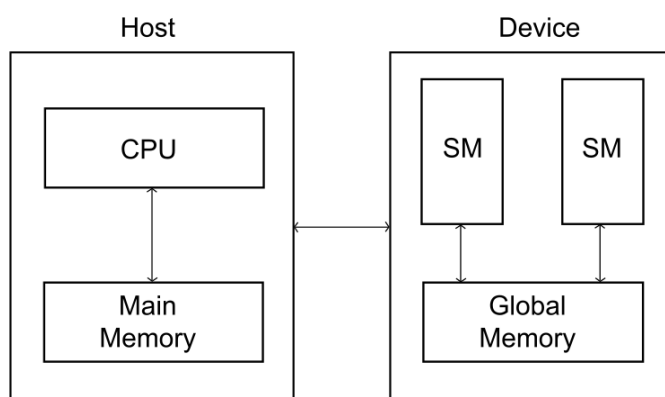


Рис. 1.13: Гетерогенная архитектура

Переменные `blockIdx`, `blockDim`, `threadIdx`, `gridDim` могут быть целыми числами или объектами типа `dim3` с полями `x`, `y`, `z`.

Перед декларацией функции указывается одно из ключевых слов:

1. `__host__` (по умолчанию). Функция может быть вызвана только с хоста и запускается на хосте.
2. `__global__`. Функция вызывается на хосте, но запускается на девайсе. Синтаксис для запуска функции следующий:

```
funcName<<<blkCount , threadIdx>>>(arguments);
```

3. `__device__`. Функция запускается с девайса на девайсе.

1.10.3 Объединенная память и явное копирование данных

Данные, с которыми работает гетерогенная программа, могут быть расположены либо в оперативной памяти хоста, либо в глобальной памяти девайса¹.

Имеется два способа организации работы с памятью в CUDA:

1. Использовать объединенную память (Unified Memory), которая представляет собой единое адресное пространство оперативной памяти хоста и глобальной памяти графической карты. При этом фактически данные могут быть как на хосте, так и на девайсе. См. рис. 1.14.
2. Явно копировать данные с хоста на девайс и обратно.

Объединенная память выделяется функцией `cudaMallocManaged`, освобождается `cudaFree`.

```
__host__ cudaError_t cudaMallocManaged(
    void** ptr ,                /* out */
    size_t size ,               /* in */
    int flags ,                 /* in */
);
```

¹Случай, когда данные расположены в регистрах девайса или в общей памяти SM, будет рассмотрен далее

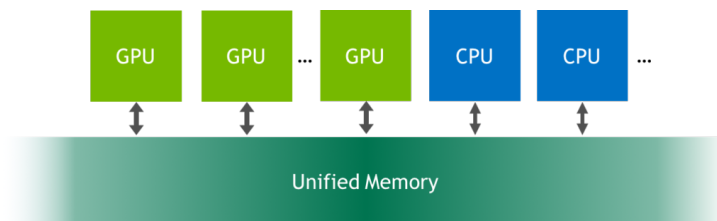


Рис. 1.14: Объединенная память

Последний аргумент позволяет указать, какие потоки будут иметь доступ к выделяемой памяти (по умолчанию с любого).

Также можно создать глобальную переменную, в начале объявления указав слово `__managed__`. В таком случае переменная будет также относиться к объединенной памяти.

Для того, чтобы явно выделить память на девайсе, используется функция `cudaMalloc`, освобождается также `cudaFree`.

```
__host__ __device__ cudaError_t cudaMalloc(
    void** ptr,          /*out*/
    size_t size          /*in*/
);

__host__ __device__ cudaError_t cudaFree(void* ptr);
```

Копирование осуществляется с помощью функции `cudaMemcpy`:

```
__host__ cudaError_t cudaMemcpy(
    void *dest,          /*out*/
    const void* source,  /*in*/
    size_t count,        /*in*/
    cudaMemcpyKind kind  /*in*/
);
```

Переменная `kind` может принимать одно из двух значений:

1. `cudaMemcpyHostToDevice`.
2. `cudaMemcpyDeviceToHost`.

1.10.4 Сложение векторов

```

#include <stdio.h>
#include <cuda.h>

__global__ void Vec_sum(float x[], float y[], float z[], int n)
{
    int my_idx = blockDim.x * blockIdx.x + threadIdx.x;
    if (my_idx < n)
        z[my_idx] = x[my_idx] + y[my_idx];
}

int main(int argc, char *argv[])
{
    // Размерность вектора
    int n = strtol(argv[1], NULL, 10);
    // Количество блоков
    int blk_ct = strtol(argv[2], NULL, 10);
    // Количество потоков в каждом блоке
    int th_per_blk = strtol(argv[3], NULL, 10);

    // Выделить память на хосте
    float *x, *y, *z;
    cudaMallocManaged(&x, n);
    cudaMallocManaged(&y, n);
    cudaMallocManaged(&z, n);

    for (int i = 0; i < n; i++)
    {
        x[i] = ((double)rand()) / RAND_MAX;
        y[i] = ((double)rand()) / RAND_MAX;
    }

    for (int i = 0; i < n; i++)
    {
        printf("%f_", x[i]);
    }
    printf("\n");
    for (int i = 0; i < n; i++)
    {
        printf("%f_", y[i]);
    }
    printf("\n");
}

```

```

// Запускаем ядро (kernel)
Vec_sum<<<blk_ct, th_per_blk>>>(x, y, z, n);

// Ждем, пока потоки завершат работу
cudaDeviceSynchronize();

for (int i = 0; i < n; i++)
{
    printf("%f_", z[i]);
}
printf("\n");

cudaFree(x);
cudaFree(y);
cudaFree(z);

return 0;
}

```

Листинг 1.8: Сложение векторов (явное копирование данных с хоста на девайс и обратно)

```

#include <stdio.h>
#include <cuda.h>

__global__ void Vec_sum(float x[], float y[], float z[], int n)
{
    int my_idx = blockDim.x * blockIdx.x + threadIdx.x;
    if (my_idx < n)
        z[my_idx] = x[my_idx] + y[my_idx];
}

int main(int argc, char *argv[])
{
    // Размерность векторов
    int n = strtol(argv[1], NULL, 10);
    // Количество блоков
    int blk_ct = strtol(argv[2], NULL, 10);
    // Количество потоков в каждом блоке
    int th_per_blk = strtol(argv[3], NULL, 10);

```

```
// Выделить память на хосте
float *hx, *hy, *hz;
hx = (float *)malloc(n * sizeof(float));
hy = (float *)malloc(n * sizeof(float));
hz = (float *)malloc(n * sizeof(float));

// Выделить память на девайсе (GPU)
float *dx, *dy, *dz;
cudaMalloc(&dx, n * sizeof(float));
cudaMalloc(&dy, n * sizeof(float));
cudaMalloc(&dz, n * sizeof(float));

for (int i = 0; i < n; i++)
{
    hx[i] = ((double)rand()) / RAND_MAX;
    hy[i] = ((double)rand()) / RAND_MAX;
}

// Копируем память с хоста на девайс
cudaMemcpy(dx, hx,
n*sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(dy, hy,
n*sizeof(float), cudaMemcpyHostToDevice);

for (int i = 0; i < n; i++)
{
    printf("%f_", hx[i]);
}

printf("\n");

for (int i = 0; i < n; i++)
{
    printf("%f_", hy[i]);
}

printf("\n");

// Запускаем ядро (kernel)
```

```
Vec_sum<<<blk_ct, th_per_blk>>>(dx, dy, dz, n);

// Ждем, пока потоки завершат работу
cudaDeviceSynchronize();

cudaMemcpy(hz, dz, n*sizeof(float),
cudaMemcpyDeviceToHost);

for (int i = 0; i < n; i++)
{
    printf("%f_", hz[i]);
}

printf("\n");

cudaFree(dx);
cudaFree(dy);
cudaFree(dz);
free(hx);
free(hy);
free(hz);

return 0;
}
```

1.11 CUDA: варп шафл и общая память

1.12 CUDA: синхронизация потоков, банки памяти

Глава 2

Семинары

2.1 Pthreads: создание и завершение потоков

Pthreads (POSIX Threads) — стандарт POSIX-реализации потоков. Реализации данного API существуют для большого числа UNIX-подобных ОС (GNU/Linux, Solaris, FreeBSD, OpenBSD, NetBSD, OS X).

Для использования в ОС Windows рекомендуется установить WSL (Windows Subsystem for Linux) — тонкий слой виртуализации, который позволяет запускать приложения Linux, не покидая привычного окружения Windows. Для установки WSL нужно выполнить следующую команду в PowerShell (запуск от имени администратора):

```
wsl --install
```

После завершения установки запустите программу Ubuntu — это и есть терминал Linux.

Логика управления потоками в pthreads в целом соответствует таковой при работе с библиотекой `thread` в C++. Отметим, что реализация потоков в C++ в Unix-системах существенно опирается на вызовы функций из pthreads. Библиотека pthreads предоставляет более низкоуровневый доступ к управлению процессом выполнения потоков, что позволяет добиться более высокой производительности. Поэтому данная библиотека широко используется для создания систем реального времени, высокочастотной торговли (HFT) и пр.

Напишем многопоточную версию «Hello, world!».

Листинг 2.1: Многопоточный «Hello, world»

```
1 #include<stdio.h>
2 #include<stdlib.h>
3 #include<pthread.h>
4
5 int thread_count;
6
7 void* routine(void* rank)
8 {
9     long my_rank = (long)rank;
10    printf("Hello_from_thread_%ld_of_%d\n",
11          my_rank, thread_count);
12    return NULL;
13 }
14
15 int main(int argc, char** argv)
16 {
17    thread_count = strtol(argv[1], NULL, 10);
```

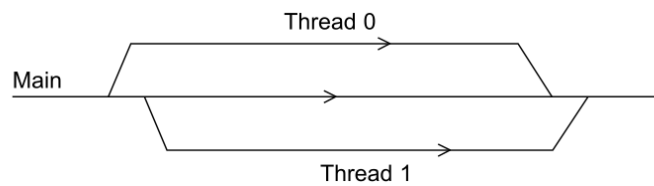


Рис. 2.1: Создание и завершение потоков

```

18 pthread_t* thread_handles =
19     malloc(thread_count * sizeof(pthread_t));
20
21 for(long i = 0; i < thread_count; i++){
22     int err = pthread_create(&thread_handles[i], NULL,
23         routine, (void*) i);
24     if(err != 0)
25         perror("Error");
26 }
27
28 printf("Hello_from_main_thread\n");
29
30 for(int i= 0; i < thread_count; i++)
31     pthread_join(thread_handles[i], NULL);
32
33 free(thread_handles);
34
35 return 0;
36 }

```

Создание потока.

```

pthread_create(
    pthread_t*          thread_p,          /*out*/
    const pthread_attr_t* attr_p,          /*in*/
    void*               (*routine) (void*), /*in*/
    void*               args               /*in*/
);

```

Первый аргумент — это дескриптор потока, куда функция `pthread_create` запишет данные конкретного создаваемого потока. Гарантируется, что в объекте типа `pthread_t` содержится достаточно информации, чтобы однозначно идентифицировать поток.

Дескрипторы потока можно использовать, например, чтобы понять, соответствуют ли два дескриптора одному или разным потокам

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

Можно получить дескриптор текущего потока

```
pthread_t pthread_self(void);
```

Второй аргумент функции `pthread_create` — атрибуты потока. Эта структура содержит параметры планировщика, размер стека и другие свойства (подробности что и как задать см. тут).

Третий аргумент — это указатель на функцию, запускаемую на потоке.

Четвертый аргумент — параметры, передаваемые в функцию.

Завершение потока

```
int pthread_join(
    pthread_t      thread_p,      /*in*/
    void**         p_arg          /*out*/
);

int pthread_detach(
    pthread_t      thread_p,      /*in*/
);

int pthread_kill(
    pthread_t      thread_p,      /*in*/
    int            sig            /*in*/
);

int pthread_cancel(
    pthread_t      thread_p      /*in*/
);

void pthread_exit(
    void**         retval         /*out*/
);
```

Функция `pthread_join` ждет, когда все потоки завершат свою работу. Далее она помещает в `&p_arg` возвращаемое потоком значение и очищает выделенные потоку ресурсы. Таким образом, когда программы покидают строку 31 программы 2.1, то мы можем быть уверены в том, что все потоки завершили свою работу, ресурсы, занятые потоками, освобождены,

а возвращаемые значения успешно сохранены (если возникла ошибка в процессе, то функция вернет ее код).

Функция `pthread_detach`, напротив, ничего не будет ждать и возвращать из потока. Она просто пометит поток как «отсоединенный» и больше ничего не будет делать. Поток сам завершит свою работу и выделенные на него ресурсы будут возвращены операционной системе. Отличие `pthread_detach` от `pthread_join` можно проиллюстрировать, сравнив результат работы программ 2.1 и 2.2 при большом количестве потоков (например, 100000 или больше). Первая программа выдаст ошибку «Cannot allocate memory», т.к. все создаваемые потоки «живут» вплоть до 31 строки, поэтому память рано или поздно исчерпается. Что же касается второй программы (листинг 2.2), то каждый вновь созданный поток функцией `detach` тут же «отсоединяется», и по завершению своей работы тут же освобождает ресурсы. Поэтому вторая программа будет работать при любом количестве потоков.

Листинг 2.2: Отличие join от detach

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>

int thread_count;

void* routine(void* rank)
{
    long my_rank = (long)rank;
    printf("Hello_from_thread_%ld_of_%d\n",
        my_rank, thread_count);
    return NULL;
}

int main(int argc, char** argv)
{
    thread_count = strtol(argv[1], NULL, 10);

    pthread_t* thread_handles = malloc(thread_count *
        sizeof(pthread_t));

    for(long i =0; i<thread_count; i++){
        int err = pthread_create(&thread_handles[i], NULL,
```

```

        routine , (void*) i );
pthread_detach(thread_handles[i]);
if(err != 0)
    perror("Error");
}

printf("Hello_from_main_thread\n");

free(thread_handles);

return 0;
}

```

Функция `pthread_kill` отправляет потоку сигнал, который передается ей в качестве второго аргумента. Константы, соответствующие передаваемым кодам, содержатся в заголовочном файле `signal.h`. Для завершения потока можно передавать константу `SIGQUIT`, при этом поток будет «убит» незамедлительно.

Функция `pthread_exit` завершает текущий поток (тот, из которого она была вызвана), возвращая из него значение `&retval`.

Функция `pthread_cancel` отправляет сигнал завершения потоку, дескриптор которого передается в качестве аргумента. При этом поток может как отреагировать на этот сигнал и завершиться (если он отменяемый), либо проигнорировать его (если он неотменяемый). По умолчанию процесс является отменяемым. Чтобы задать отменяемость процесса, используется функция

```

int pthread_setcancelstate(
    int     state,           /*in*/
    int     *oldstate       /*out*/
);

```

Первый аргумент — это константа, значение которой `PTHREAD_CANCEL_ENABLE` или `PTHREAD_CANCEL_DISABLE`.

Во втором аргументе можно сохранить текущий статус потока.

Если поток отменяемый, то посылаемый сигнал отмены будет исполнен либо асинхронно, либо отложено (по умолчанию). Асинхронное исполнение означает исполнение в любой момент (как правило, немедленно). Отложенное исполнение означает, что поток завершится, как только встретит одну из команд, являющихся точками отмены. Полный список таких команд см. здесь.

Конкретный тип отмены задается аналогично, с помощью функции

```
int pthread_setcanceltype(
    int    type,           /*in*/
    int    *oldtype       /*out*/
);
```

Первый аргумент — это одна из двух констант
 PTHREAD_CANCEL_DEFERRED
 PTHREAD_CANCEL_ASYNCHRONOUS

Во втором аргументе сохраняется текущее значение.

Следующая программа (листинг 2.3) демонстрирует передачу структуры в качестве аргумента для потока, а также получение возвращаемого потоком значения.

Листинг 2.3: Передача и возвращение структур из потока

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <signal.h>

int thread_count;

struct S
{
    long rank;
    char* title;
};

void *routine(void *s)
{
    struct S* my_s = (struct S*)s;
    printf("Hello_from_thread_%ld_of_%d, Title: %s\n",
        my_s->rank, thread_count, my_s->title);
    struct S* ret = malloc(sizeof(struct S));
    ret->rank = my_s->rank;
    ret->title = "return";
    return ret;
}

int main(int argc, char **argv)
```

```

{

    thread_count = strtol(argv[1], NULL, 10);

    pthread_t *thread_handles = malloc(thread_count * sizeof(pthread_t));

    for (long i = 0; i < thread_count; i++)
    {
        struct S* s = malloc(sizeof(struct S));
        s->rank = i;
        s->title = "Title";
        int err = pthread_create(&thread_handles[i],
                                NULL, routine, (void *)s);
        if (err != 0)
            perror("Error");
    }

    printf("Hello_from_main_thread\n");

    for (int i = 0; i < thread_count; i++){
        struct S* s;
        pthread_join(thread_handles[i], (void**)&s);
        printf("Return:_rank=%ld,_title=%s\n", s->rank, s->title);
        free(s);
    }

    free(thread_handles);

    return 0;
}

```

Время	Поток 1	Поток 2
0	Взять операнды	—
0	Передвинуть точку	Взять операнды
0	Сложить мантиссы	Передвинуть точку
0	Округлить	Сложить мантиссы
0	Записать результат	Округлить
0	—	Записать результат

Таблица 2.1: Механизм возникновения гонки

2.2 Pthreads: мьютексы и семафоры

2.2.1 Состояние гонки. Критические секции

Разложим функцию $\arctg$ в ряд Тейлора в окрестности точки $x = 0$:

$$\arctg x = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots \quad (2.1)$$

Подставляя в (2.1) $x = 1$ и учитывая, что $\arctg 1 = \frac{\pi}{4}$, получаем следующее выражение для вычисления числа π :

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots \right) \quad (2.2)$$

В листинге 2.4 представлена неудачная попытка посчитать сумму (2.2). Результат работы программы 2.4 при запуске с несколькими потоками оказывается недетерминированным. Причина в состоянии гонки в строке 22.

Продемонстрируем механизм возникновения состояния гонки на примере. Предположим, что начальное значение переменной `sum` равно 10. Пусть работают два потока и первый увеличивает значение `sum` на 2, а второй уменьшает на 5. Тогда после выполнения строки 22 обоими потоками в переменной `sum` должно храниться число 7. Однако если первый поток начнет выполнять присваивание раньше, чем второй, а закончит после того, как второй начнет, но до того, как закончит, то второй поток «затрет» результат работы первого потока (см. таблицу 2.1. Тогда легко видеть, что в переменной `sum` в рассматриваемом случае будет лежать число 5.

Листинг 2.4: Неудачная попытка посчитать число π

```

1
2 double sum = 0;
```

```

3
4 void *routine(void *rank){
5
6     long my_rank = (long)rank;
7     double factor;
8     int my_n = n / thread_count;
9     int my_first_i = my_rank * my_n;
10    int my_last_i = (my_rank + 1) * my_n - 1;
11    if(my_first_i % 2 == 0)
12        factor = 1.0;
13    else
14        factor = -1.0;
15
16    for (int i = my_first_i; i < my_last_i; i++, factor *= -1)
17    {
18        sum += factor / (2 * i + 1);
19    }
20
21    return NULL;
22
23 }
```

Таким образом, строка 22 в листинге 2.4 является **критической секцией**. Так называется фрагмент программы, в случае одновременного выполнения которого несколькими потоками может возникнуть гонка. Следовательно, необходим механизм, исключающий одновременное выполнение несколькими потоками критической секции. Далее рассмотрим два таких механизма: холостой цикл и мьютексы.

Использование холостого цикла показано в листинге 2.5.

Листинг 2.5: Холостой цикл

```

int flag = 0;
void *routine(void *rank)
{

    long my_rank = (long)rank;
    double factor;
    int my_n = n / thread_count;
    int my_first_i = my_rank * my_n;
    int my_last_i = (my_rank + 1) * my_n - 1;
    if (my_first_i % 2 == 0)
        factor = 1.0;
```

```

    else
        factor = -1.0;

    double my_sum = 0.0;

    for (int i = my_first_i; i < my_last_i; i++, factor *= -1)
    {
        my_sum += factor / (2 * i + 1);
    }
    while (flag != my_rank);
    sum += my_sum;
    flag++;

    return NULL;
}

```

Второй механизм исключения одновременного выполнения критической секции — мьютекс — показан в листинге 2.6. Мьютекс — это примитив синхронизации, которому в `pthread`s соответствует тип данных `pthread_mutex_t`.

Мьютекс инициализируется с помощью функции

```

int pthread_mutex_init(
    pthread_mutex_t*      mutex,      /*out*/
    pthread_mutexattr_t*  attr_p     /*in*/
);

```

Второй аргумент — указатель на атрибуты мьютекса, которые позволяют задать его тип с помощью функции `pthread_mutexattr_settype`:

```

int pthread_mutexattr_settype(
    pthread_mutexattr_t  *attr,      /*out*/
    int                  type        /*in*/
);

```

Некоторые типы мьютексов (второй аргумент `pthread_mutexattr_settype`):

```

PTHREAD_MUTEX_NORMAL
PTHREAD_MUTEX_ERRORCHECK
PTHREAD_MUTEX_RECURSIVE

```

Две основные функции для работы с мьютексами:

```

int pthread_mutex_lock(
    pthread_mutex_t* mutex /*in/out*/
);

```

Время	Поток 1	Поток 2
0	pthread_mutex_lock(&m1)	pthread_mutex_lock(&m2)
1	pthread_mutex_lock(&m2)	pthread_mutex_lock(&m1)

Таблица 2.2: Дэдлок

```
int pthread_mutex_unlock(
    pthread_mutex_t* mutex /*in/out*/
);
```

Когда происходит выполнение функции `pthread_mutex_lock`, то переданный ей в качестве аргумента мьютекс будет помечен как «закрытый», а его владельцем будет вызвавший функцию поток. Если поток 1 закрыл мьютекс, а поток 2 попытается его еще раз закрыть, то поток 2 будет ожидать в функции `pthread_mutex_lock`, пока поток 1 не откроет его вызовом `pthread_mutex_unlock`. После этого поток 2 закроет мьютекс и продолжит выполнение кода, став новым владельцем мьютекса. Отметим, что **открыть мьютекс может только тот поток, который его закрыл** (который им владеет в данный момент). При попытке открыть мьютекс, закрытый другим потоком, поведение программы не определено.

Дэдлок (deadlock) возникает, когда потоки вынуждены ждать получения мьютекса бесконечно долго. Пример см. в таблице 2.2.

Освободить ресурсы, выделенные для мьютекса, можно с помощью функции

```
int pthread_mutex_destroy(
    pthread_mutex_t* mutex /*in, out*/
);
```

Листинг 2.6: Использование мьютекса

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int thread_count;

int n;
double sum = 0;

int flag = 0;
```

```

pthread_mutex_t mutex;

void *routine(void *rank)
{
    long my_rank = (long)rank;
    double factor;
    int my_n = n / thread_count;
    int my_first_i = my_rank * my_n;
    int my_last_i = (my_rank + 1) * my_n - 1;
    if (my_first_i % 2 == 0)
        factor = 1.0;
    else
        factor = -1.0;

    double my_sum = 0.0;

    for (int i = my_first_i; i < my_last_i; i++, factor *= -1)
    {
        my_sum += factor / (2 * i + 1);
    }
    pthread_mutex_lock(&mutex);
    sum += my_sum;
    pthread_mutex_unlock(&mutex);

    return NULL;
}

int main(int argc, char **argv)
{
    thread_count = strtol(argv[1], NULL, 10);
    n = strtol(argv[2], NULL, 10);

    pthread_t *thread_handles =
        malloc(thread_count * sizeof(pthread_t));

    pthread_mutex_init(&mutex, NULL);

    for (long i = 0; i < thread_count; ++i)
        pthread_create(&thread_handles[i],

```

```

        NULL, routine , (void *)i);

    for (int i = 0; i < thread_count; ++i)
        pthread_join(thread_handles[i], NULL);

    printf("pi=%lf\n", 4 * sum);

    pthread_mutex_destroy(&mutex);
    free(thread_handles);

    return 0;
}

```

Отличия типов мьютексов друг от друга:

1. **PTHREAD_MUTEX_NORMAL**. Мьютекс по умолчанию. Из соображений скорости работы не осуществляется контроль ошибок. Так, попытка потока залочить уже закрытый им же мьютекс, равно как и попытка открыть закрытый кем-то другим (или открытый) мьютекс, приведет к неопределенному поведению.
2. **PTHREAD_MUTEX_ERRORCHECK**. Мьютекс со встроенной проверкой на ошибки. Если поток попытается закрыть мьютекс, который он же закрывал, то вернется ошибка. Также ошибка будет возвращена, если поток попытается открыть мьютекс, который он не закрывал (или который открыт).
3. **PTHREAD_MUTEX_RECURSIVE**. В отличие от предыдущих двух мьютексов, данный тип допускает повторное закрытие тем же потоком без ошибок или неопределенного поведения. При этом происходит подсчет закрытий и открытий.

Отличие первых двух типов мьютексов между собой демонстрирует листинг 2.7. Программа выведет сообщения `Operation not permitted`. Если в строке 35 заменить тип мьютекса на `PTHREAD_MUTEX_NORMAL`, то сообщений об ошибках не будет.

Листинг 2.7: Демонстрация отличий мьютекса с проверкой ошибок от обычного

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <semaphore.h>

```

```
5 #include <errno.h>
6 #define MSG_SIZE 100
7
8 pthread_mutex_t m;
9
10 int thread_count;
11
12 char **messages;
13
14 void *routine(void *rank)
15 {
16
17     long my_rank = (long)rank;
18     errno = pthread_mutex_unlock(&m);
19     if (errno != 0)
20         perror("Error");
21
22     return NULL;
23 }
24
25 int main(int argc, char **argv)
26 {
27     thread_count = strtol(argv[1], NULL, 10);
28
29     messages = malloc(thread_count * sizeof(char *));
30
31     pthread_t *thread_handles =
32         malloc(thread_count * sizeof(pthread_t));
33
34     pthread_mutexattr_t attr;
35     pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_ERRORCHECK);
36
37     pthread_mutex_init(&m, &attr);
38     pthread_mutex_lock(&m);
39
40     for (long i = 0; i < thread_count; ++i)
41         pthread_create(&thread_handles[i], NULL, routine, (void *)i);
42
43     for (int i = 0; i < thread_count; ++i)
44         pthread_join(thread_handles[i], NULL);
45 }
```

```

46     free ( thread_handles );
47     pthread_mutex_destroy(&m);
48
49     return 0;
50 }

```

2.2.2 Семафоры

Рассмотрим задачу синхронизации типа «производитель — потребитель», простейшая формулировка которой требует от потоков передавать сообщения друг другу по кругу. Первая попытка такой программы показана в листинге 2.8. Результат работы программы предсказуем: некоторые потоки попадут в 20 строку раньше, чем предшествующие потоки выполнят присваивание в 19 строке. Поэтому некоторые потоки не смогут вывести предназначенные для них сообщения.

Листинг 2.8: Первая попытка потоков оставлять сообщения друг другу

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4
5  int thread_count;
6
7  #define MSG_SIZE 100
8
9  char** messages;
10
11 void *routine(void *rank)
12 {
13
14     long my_rank = (long)rank;
15     int dest = (my_rank + 1) % thread_count;
16     int source = (my_rank + thread_count - 1) % thread_count;
17     char* mes = malloc(MSG_SIZE * sizeof(char));
18     sprintf(mes, "Hello_from_%ld_to_%d", my_rank, dest);
19     messages[dest] = mes;
20     if(messages[my_rank] != NULL)
21         printf("Thread_%ld->%s\n", my_rank, messages[my_rank]);
22     else
23         printf("Thread_%ld->_No_messages\n", my_rank);
24     return NULL;

```

```

25 }
26
27 int main(int argc, char **argv)
28 {
29     thread_count = strtol(argv[1], NULL, 10);
30
31     messages = malloc(thread_count * sizeof(char*));
32
33     pthread_t *thread_handles =
34         malloc(thread_count * sizeof(pthread_t));
35
36     for (long i = 0; i < thread_count; ++i)
37         pthread_create(&thread_handles[i], NULL,
38             routine, (void *)i);
39
40     for (int i = 0; i < thread_count; ++i)
41         pthread_join(thread_handles[i], NULL);
42
43     free(thread_handles);
44
45     for(int i=0; i<thread_count; i++)
46         free(messages[i]);
47     free(messages);
48
49     return 0;
50 }

```

Корректная реализация алгоритма возможна с использованием семафоров.

Семафор — это переменная типа `sem_t`, которая представляет собой беззнаковое целое число.

Инициализация семафора

```

int sem_init(
    sem_t*      s_p,      /*out*/
    int         shared,    /*int*/
    int         ini_v     /*int*/
);

```

Второй аргумент — 0, если семафор общий для всех потоков, или 1, если нет. Третий аргумент — начальное значение семафора.

Освобождение ресурсов семафора

```
int sem_init(  
    sem_t*      s_p,      /*in/out*/  
);
```

Две основные команды для работы с семафорами — это поднять семафор, то есть увеличить на единицу (**post**), или опустить, то есть уменьшить на единицу (**wait**).

```
int sem_init(  
    sem_t*      s_p,      /*in/out*/  
);  
  
int sem_wait(  
    sem_t*      s_p,      /*in/out*/  
);
```

Если текущее значение семафора единица или выше, то вызов функции **wait** просто уменьшает его на единицу. Если значение семафора равно 0, то вызов функции **wait** приведет к блокировке потока. Поток не покинет функцию **wait**, пока другой поток не выполнит **post** для этого же семафора. Тогда значение семафора станет равным единице, функция **wait** в заблокированном потоке опустит семафор и поток продолжит выполнять дальнейшие инструкции.

Пример использования семафоров для решения вышеставленной задачи см. в листинге 2.9.

Листинг 2.9: Использование семафоров

```
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
#include <semaphore.h>  
#define MSG_SIZE 100  
  
sem_t *sems;  
  
int thread_count;  
  
char **messages;  
  
void *routine(void *rank)  
{
```

```

    long my_rank = (long)rank;
    int dest = (my_rank + 1) % thread_count;
    int source = (my_rank + thread_count - 1) % thread_count;
    char *mes = malloc(MSG_SIZE * sizeof(char));
    sprintf(mes, "Hello_from_%ld_to_%ld", my_rank, dest);
    messages[dest] = mes;
    sem_post(&sems[dest]);
    sem_wait(&sems[my_rank]);
    printf("Thread_%ld->%s\n", my_rank, messages[my_rank]);
    return NULL;
}

int main(int argc, char **argv)
{
    thread_count = strtol(argv[1], NULL, 10);

    messages = malloc(thread_count * sizeof(char *));

    pthread_t *thread_handles =
        malloc(thread_count * sizeof(pthread_t));

    sems = malloc(thread_count * sizeof(sem_t));

    for (int i = 0; i < thread_count; i++)
        sem_init(&sems[i], 0, 0);

    for (long i = 0; i < thread_count; ++i)
        pthread_create(&thread_handles[i],
            NULL, routine, (void *)i);

    for (int i = 0; i < thread_count; ++i)
        pthread_join(thread_handles[i], NULL);

    free(thread_handles);

    for (int i = 0; i < thread_count; i++)
        sem_destroy(&sems[i]);

    for (int i = 0; i < thread_count; i++)
        free(messages[i]);
    free(messages);
}

```

```
    free (sems);  
  
    return 0;  
}
```

Бинарный семафор можно использовать в качестве мьютекса. Основное отличие заключается в том, что для семафоров нет понятия владения. Поднять и опустить семафор может любой поток. Что же касается мьютекса, то открыл его может только тот поток, который закрыл (завладел).

Небинарный семафор можно использовать, например, для контроля ограниченных ресурсов.

2.3 Pthreads: потокобезопасность

Задача. Написать параллельный токенизатор: программу, которая из полученного текста выделит отдельные слова.

Будем считать, что нулевой поток читает самую первую строку, потом первый поток читает вторую и т.д. Таким образом, процесс чтения строк из файла не является параллельным и более того организован упорядочено. После чтения все потоки более или менее одновременно будут обрабатывать свои строки. Если количество строк больше количества потоков, то потом процесс продолжится: поток с рангом 0 прочитает строку номер `thread_count + 1` и т.д.

В листинге 2.10 предложена первая попытка решения поставленной задачи. В основе работы программы — использование функции `strtok`:

```
char* strtok(
    char* string ,           /*in/out*/
    char* separators         /*out*/
)
```

Данная функция работает следующим образом: при первом запуске в качестве первого аргумента передаем строку, которую хотим разбить на токены. При этом функция вернет первый токен. При втором и последующем запусках передаем первым аргументом `NULL`, тогда функция вернет второй и последующие токены той же строки.

Листинг 2.10: Многопоточная токенизация (первая попытка)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define MAX_LEN 100

sem_t *sems;

int thread_count;

void *routine(void *rank)
{
    char *p = NULL;
    long my_rank = (long)rank;
```

```

char my_line[MAX_LEN];
char *token = NULL;
sem_wait(&sems[my_rank]);
p = fgets(my_line, MAX_LEN, stdin);
sem_post(&sems[(my_rank + 1) % thread_count]);

while (p != NULL)
{
    printf("Thread_%ld->_line_=%s\n", my_rank, my_line);
    token = strtok(my_line, "_\\t\\n");
    while (token != NULL)
    {
        printf("Thread_%ld->_token=%s\n", my_rank, token);
        token = strtok(NULL, "_\\t\\n");
    }
    sem_wait(&sems[my_rank]);
    p = fgets(my_line, MAX_LEN, stdin);
    sem_post(&sems[(my_rank + 1) % thread_count]);
}

return NULL;
}

int main(int argc, char *argv[])
{
    thread_count = strtol(argv[1], NULL, 10);
    pthread_t *thread_handles =
        malloc(thread_count * sizeof(pthread_t));

    sems = malloc(thread_count * sizeof(sem_t));
    sem_init(&sems[0], 0, 1);
    for (int i = 1; i < thread_count; i++)
        sem_init(&sems[i], 0, 0);

    for (long i = 0; i < thread_count; i++)
        pthread_create(&thread_handles[i], NULL, routine, (void *)i);
    for (long i = 0; i < thread_count; i++)
        pthread_join(thread_handles[i], NULL);

    for (int i = 0; i < thread_count; i++)
        sem_destroy(&sems[i]);
}

```

```

    free(thread_handles);
    free(sems);
    return 0;
}

```

Результат работы программы: при некоторых запусках будет выводиться корректный результат. При других запусках какие-то токены потеряются. Причина в следующем: функция `str_tok` использует в качестве буфера `static` переменную, т.е. переменную, которая не меняет своего значение от запуска к запуску функции и, конечно, относится к общей памяти. В результате возникает состояние гонки.

Говорят, что функции наподобие `str_tok` не являются потокобезопасными, или не реинтерабельны.

Чтобы программа в листинге 2.10 работала корректно, необходимо использовать потокобезопасную (или реинтерабельную) версию `str_tok`, которая называется `str_tok_r`. Она принимает дополнительный аргумент: указатель на строку-буфер, которая должна быть расположена в локальной памяти потока. См. листинг 2.11.

Листинг 2.11: Потокобезопасная токенизация

```

void *routine(void *rank)
{
    char *p = NULL;
    long my_rank = (long)rank;
    char my_line[MAX_LEN];
    char *token = NULL;
    sem_wait(&sems[my_rank]);
    p = fgets(my_line, MAX_LEN, stdin);
    sem_post(&sems[(my_rank + 1) % thread_count]);
    char *buf[MAX_LEN];

    while (p != NULL)
    {
        printf("Thread_%ld->_line_=%s\n", my_rank, my_line);
        token = strtok_r(my_line, "\\t\\n", buf);
        while (token != NULL)
        {
            printf("Thread_%ld->_token=%s\n", my_rank, token);
            token = strtok_r(NULL, "\\t\\n", buf);
        }
    }
}

```

```
    sem_wait(&sems[my_rank]);  
    p = fgets(my_line, MAX_LEN, stdin);  
    sem_post(&sems[(my_rank + 1) % thread_count]);  
}  
  
return NULL;  
}
```

2.4 Pthreads: барьеры и условные переменные

Барьер — это фрагмент кода, который характеризуется тем, что ни один из потоков не выйдет из него, пока последний не зашел.

2.4.1 Холостой цикл

Самый простой способ организации барьера — это использование холостого цикла. Минусы такого подхода:

1. Значительное падение производительности в случае, когда количество запущенных потоков превышает количество физических. Это связано с тем, что проходит значительное время, прежде чем операционная система догадается, что поток ничего не делает и висит в бездействии, и отключит его, чтобы дать возможность поработать другим потокам.
2. Компилятор не знает, что мы пишем многопоточную программу. Если включить оптимизацию (использовать флаг `-O2`), то компилятор может переставить некоторые строки, которые, с его точки зрения, не зависят друг от друга, и тогда программа перестанет работать.
3. Мы не можем синхронно обнулить `counter`, поэтому эта переменная является одноразовой. Нам придется создавать массив счетчиков, если барьеров несколько.

Листинг 2.12: Организация барьера с помощью холостого цикла

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include "timer.h"

#define NTHREADS 4
#define NBARRIERS 10000
pthread_mutex_t mutex;
int counter[NBARRIERS];

void *routine(void *rank)
{
```

```

    for (int j = 0; j < NBARRIERS; j++)
    {
        pthread_mutex_lock(&mutex);
        counter[j]++;
        pthread_mutex_unlock(&mutex);
        while(counter[j] != NTHREADS);
    }
    return NULL;
}

int main()
{
    pthread_t thread_handles[NTHREADS];
    pthread_mutex_init(&mutex, NULL);
    double start, end;
    GET_TIME(start);
    for (int i = 0; i < NTHREADS; i++)
        pthread_create(&thread_handles[i], NULL, routine, NULL);

    for (int i = 0; i < NTHREADS; i++)
        pthread_join(thread_handles[i], NULL);
    GET_TIME(end);
    printf("%e\n", end - start);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

2.4.2 Семафоры

Листинг 2.13: Организация барьера с помощью семафоров

```

1
2 #include <pthread.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include "timer.h"
6 #include <semaphore.h>
7
8 #define NTHREADS 4
9 #define NBARRIERS 10000
10 pthread_mutex_t mutex;

```

```
11 sem_t sem_barrier[NBARRIERS];
12 sem_t sem_count;
13 int counter = 0;
14
15 void *routine(void *rank)
16 {
17     for (int j = 0; j < NBARRIERS; j++)
18     {
19         sem_wait(&sem_count);
20         if (counter == NTHREADS - 1)
21         {
22             counter = 0;
23             sem_post(&sem_count);
24             for (int i = 0; i < NTHREADS - 1; i++)
25                 sem_post(&sem_barrier[j]);
26         }
27         else
28         {
29             counter++;
30             sem_post(&sem_count);
31             sem_wait(&sem_barrier[j]);
32         }
33     }
34     return NULL;
35 }
36
37 int main()
38 {
39     pthread_t thread_handles[NTHREADS];
40     pthread_mutex_init(&mutex, NULL);
41     for (int i=0; i<NBARRIERS; i++)
42         sem_init(&sem_barrier[i], 0, 0);
43     sem_init(&sem_count, 0, 1);
44     double start, end;
45     GET_TIME(start);
46     for (int i = 0; i < NTHREADS; i++)
47         pthread_create(&thread_handles[i], NULL, routine, NULL);
48
49     for (int i = 0; i < NTHREADS; i++)
50         pthread_join(thread_handles[i], NULL);
51     GET_TIME(end);
```

```

52     printf("%e\n", end - start);
53     for(int i=0; i<NBARRIERS; i++)
54         sem_destroy(&sem_barrier[i]);
55     sem_destroy(&sem_count);
56     pthread_mutex_destroy(&mutex);
57     return 0;
58 }

```

По прохождению барьера в программе 2.13 обнуляется `counter`, а семафор `sem_count` опять принимает значение 1. Однако семафор `barrier_sem` может не занулиться. Именно, может реализоваться следующий сценарий: потоки, все, кроме последнего, зашедшего в барьер, уснут в строке 31. Операционная система, заметив бездействие потоков, временно прекратит их работу. Последний поток в 25 строке поднимет семафор, пробуждая зависшие потоки. Однако, если потоки отключены от планировщика задач, то они не увидят поднятых семафоров. Тогда последний поток, подняв семафор некоторое количество раз, продолжит работу, а значение `barrier_sem` будет отлично от нуля. В этом случае последний поток продолжит работу, дойдет до второго барьера, а т.к. `barrier_sem` не занулен, то этот поток пойдет дальше второго барьера, к третьему, а зависшие в первом барьере потоки так его и не преодолеют.

Следовательно, барьерный семафор является одноразовым. Его нельзя переиспользовать.

2.4.3 Условные переменные

Для реализации барьера, которая была бы лишена выявленных выше недостатков, будем использовать новый тип данных — условную переменную, `pthread_cond_t`.

Инициализация условной переменной

```

int pthread_cond_init(
    pthread_cond_t*    cond_p,    /*out*/
    pthread_condattr_t cond_attr  /*in*/
);

```

Освобождение ресурсов, занимаемых условной переменной

```

int pthread_cond_destroy(
    pthread_cond_t*    cond_p    /*in/out*/
);

```

Условная переменная используется для того, чтобы сигнализировать потокам о том, что событие, которое символизирует данная условная переменная, произошло.

Условная переменная используется всегда в связке с мьютексом. Пример использования условной переменной для реализации барьера показан в листинге 2.14.

Обратим внимание на то, что в строке 29 вызов функции `pthread_cond_wait` поставлен в цикл `while`. Это сделано для того, чтобы отличить «пробуждение» потока вследствие срабатывания условной переменной от самопроизвольного пробуждения, вызванного другими причинами. Но скорее всего все будет хорошо работать и без цикла.

Листинг 2.14: Барьер с использованием условных переменных

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include "timer.h"
5 #include <semaphore.h>
6
7 #define NTHREADS 4
8 #define NBARRIERS 10000
9 pthread_mutex_t mutex;
10 sem_t sem_barrier[NBARRIERS];
11 sem_t sem_count;
12 int counter = 0;
13
14 pthread_cond_t cond;
15
16 void *routine(void *rank)
17 {
18     for (int j = 0; j < NBARRIERS; j++)
19     {
20         pthread_mutex_lock(&mutex);
21         counter++;
22         if (counter == NTHREADS)
23         {
24             counter = 0;
25             pthread_mutex_unlock(&mutex);
26             pthread_cond_broadcast(&cond);
27         }
28     }
29     pthread_cond_wait(&cond, &mutex);
30 }
```

```
29     while(pthread_cond_wait(&cond, &mutex) != 0);
30     /*
31     pthread_cond_wait(&cond, &mutex):
32         pthread_mutex_unlock(&mutex);
33         wait_on_signal(&cond);
34         pthread_mutex_lock(&mutex);
35     */
36     pthread_mutex_unlock(&mutex);
37 }
38 return NULL;
39 }
40
41 int main()
42 {
43     pthread_t thread_handles[NTHREADS];
44     pthread_cond_init(&cond, NULL);
45     pthread_mutex_init(&mutex, NULL);
46     double start, end;
47     GET_TIME(start);
48     for (int i = 0; i < NTHREADS; i++)
49         pthread_create(&thread_handles[i], NULL, routine, NULL);
50
51     for (int i = 0; i < NTHREADS; i++)
52         pthread_join(thread_handles[i], NULL);
53     GET_TIME(end);
54     printf("%e\n", end - start);
55     pthread_cond_destroy(&cond);
56     pthread_mutex_destroy(&mutex);
57     return 0;
58 }
```

2.5 MPI: отправка и получение сообщений

Задача. Написать свою реализацию `MPI_Reduce` для частного случая, когда `root = 0` и `MPI_Op = MPI_SUM`.

Предположим для простоты, что `comm_sz` это степень двойки.

2.5.1 Способ 1: индусятина ☺

Будем использовать переменную `divisor` для того, чтобы определить, является ли данный процесс отправителем или получателем. Значение этой переменной равно 2 и удваивается после каждой итерации.

Также введем переменную `core_difference`, которая равна одному и также удваивается после каждой итерации. С ее помощью мы будем вычислять партнера для нашего процесса.

Листинг 2.15: Первая реализация суммирования

```
void reduce_indian()
{
    int my_value = my_rank;
    int divisor = 2;
    int core_difference = 1;

    while (core_difference < comm_sz) {
        if (my_rank % divisor == 0) {
            int source = my_rank + core_difference;
            int buffer;
            MPI_Recv(&buffer, 1, MPI_INT, source, 0,
                    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            my_value += buffer;
        }
        else if (my_rank - core_difference >= 0) {
            int dest = my_rank - core_difference;
            MPI_Send(&my_value, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
            break;
        }
        core_difference *= 2;
        divisor *= 2;
    }

    if (my_rank == 0)
        printf("%d\n", my_value);
}
```

```
}
```

2.5.2 Способ 2: сигма-версия ☺

Записав в двоичном представлении ранги процессов от 0 до `comm_sz - 1` и посмотрев, как на различных стадиях изменяются номера пар процессов, легко увидеть закономерность: пары процессов на первом этапе суммирования отличаются только младшими битами, на втором этапе вторыми справа битами и т.д. Данная идея используется в листинге 2.16.

Листинг 2.16: Суммирование (более короткая версия)

```
void reduce_sigma()
{
    int my_value = my_rank;
    int mask = 1;
    int fictive_comm_sz = 1;

    while (mask < comm_sz){
        int my_partner = my_rank ^ mask;
        if (my_partner >= comm_sz){
            mask *= 2;
            continue;
        }

        if(my_rank < my_partner){
            int buff;
            MPI_Recv(&buff, 1, MPI_INT, my_partner, 0,
                    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            my_value += buff;
        }
        else if (my_rank > my_partner){
            MPI_Send(&my_value, 1, MPI_INT, my_partner,
                    0, MPI_COMM_WORLD);
            break;
        }
        mask *= 2;
    }
    if(my_rank == 0){
        printf("%d\n", my_value);
    }
}
```

2.5.3 Адаптация сигмы для случая, когда `comm_sz` не степень двойки

Введем фиктивные процессы, дополнив их общее количество до ближайшей сверху степени двойки. С этой целью вводится переменная `fictive_comm_size`. Будем отслеживать, чтобы не происходило коммуникации с фиктивными процессами. Эта идея использована в листинге 2.17.

Листинг 2.17: Короткая версия без ограничений на `comm_sz`

```
void reduce_sigma()
{
    int my_value = my_rank;
    int mask = 1;
    int fictive_comm_sz = 1;
    while (fictive_comm_sz < comm_sz){
        fictive_comm_sz *= 2;
    }
    while (mask < fictive_comm_sz){
        int my_partner = my_rank ^ mask;
        if (my_partner >= comm_sz){
            mask *= 2;
            continue;
        }

        if(my_rank < my_partner){
            int buff;
            MPI_Recv(&buff, 1, MPI_INT, my_partner, 0,
                    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            my_value += buff;
        }
        else if (my_rank > my_partner){
            MPI_Send(&my_value, 1, MPI_INT,
                    my_partner, 0, MPI_COMM_WORLD);
            break;
        }
        mask *= 2;
    }
    if(my_rank == 0){
        printf("%d\n", my_value);
    }
}
```

2.6 MPI: практика коллективных коммуникаций

Задача 1. Реализовать скалярное произведение векторов.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

void InputDim(int *n, int my_rank)
{
    if (my_rank == 0)
    {
        scanf("%d", n);
    }
    MPI_Bcast(n, 1, MPI_INT, 0, MPI_COMM_WORLD);
}

void InputVector(int *v, int n, int my_rank, int comm_sz)
{
    int *temp = calloc(n, sizeof(int));
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
        {
            temp[i] = rand() % 10;
        }
        MPI_Scatter(temp, n / comm_sz, MPI_INT, v, n / comm_sz,
                    MPI_INT, 0, MPI_COMM_WORLD);
    }
    else
        MPI_Scatter(temp, n / comm_sz, MPI_INT, v, n / comm_sz,
                    MPI_INT, 0, MPI_COMM_WORLD);

    free(temp);
}

void PrintDistributedVector(int *v, int n,
                             int my_rank, int comm_sz)
{
    int *temp = calloc(n, sizeof(int));
```

```

MPI_Gather(v, n / comm_sz, MPI_INT, temp, n / comm_sz,
MPI_INT, 0, MPI_COMM_WORLD);
if (my_rank == 0)
{
    for (int i = 0; i < n; i++)
        printf("%d_", temp[i]);
    printf("\n");
}
free(temp);
}

void DotProd(int *v1, int *v2, int local_n, int my_rank)
{
    int local_res = 0, res = 0;
    for (int i = 0; i < local_n; i++)
        local_res += v1[i] * v2[i];
    MPI_Reduce(&local_res, &res, 1, MPI_INT,
MPI_SUM, 0, MPI_COMM_WORLD);
    if (my_rank == 0)
        printf("%d\n", res);
}

int main()
{
    int comm_sz;
    int my_rank;

    MPI_Init(NULL, NULL);

    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    int n;
    InputDim(&n, my_rank);
    int *vec1, *vec2;
    vec1 = calloc(n / comm_sz, sizeof(int));
    vec2 = calloc(n / comm_sz, sizeof(int));
    InputVector(vec1, n, my_rank, comm_sz);
    InputVector(vec2, n, my_rank, comm_sz);
    PrintDistributedVector(vec1, n, my_rank, comm_sz);

```

```

    PrintDistributedVector(vec2, n, my_rank, comm_sz);
    DotProd(vec1, vec2, n / comm_sz, my_rank);

    MPI_Finalize();
}

```

Задача 2. Умножить матрицу на вектор, используя построчное разбиение матрицы.

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

void InputDim(int *n, int my_rank)
{
    if (my_rank == 0)
    {
        scanf("%d", n);
    }
    MPI_Bcast(n, 1, MPI_INT, 0, MPI_COMM_WORLD);
}

void InputVector(int *v, int n, int my_rank)
{
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
        {
            v[i] = rand() % 10;
        }
    }
    MPI_Bcast(v, n, MPI_INT, 0, MPI_COMM_WORLD);
}

void PrintDistributedVector(int *v, int n,
    int my_rank, int comm_sz)
{
    int *temp = calloc(n, sizeof(int));
    MPI_Gather(v, n / comm_sz, MPI_INT, temp, n / comm_sz,
        MPI_INT, 0, MPI_COMM_WORLD);
    if (my_rank == 0)
    {

```

```

    for (int i = 0; i < n; i++)
        printf("%d_", temp[i]);
    printf("\n");
}
free(temp);
}

void InputMatrix(int r, int c, int *mat, int my_rank, int local_r)
{
    if (my_rank == 0)
    {
        int *temp = calloc(r * c, sizeof(int));
        if (my_rank == 0)
            for (int i = 0; i < r * c; i++)
                temp[i] = rand() % 10;
        MPI_Scatter(temp, local_r * c, MPI_INT,
            mat, local_r * c, MPI_INT, 0, MPI_COMM_WORLD);
        free(temp);
    }
    else
    {
        MPI_Scatter(NULL, local_r * c, MPI_INT, mat,
            local_r * c, MPI_INT, 0, MPI_COMM_WORLD);
    }
}

void PrintDistributedMatrix(int r, int c, int *local_mat,
    int my_rank, int local_r)
{
    if (my_rank == 0)
    {
        int *temp = calloc(r * c, sizeof(int));
        MPI_Gather(local_mat, local_r * c, MPI_INT, temp,
            local_r * c, MPI_INT, 0, MPI_COMM_WORLD);
        for (int i = 0; i < r; i++)
        {
            for (int j = 0; j < c; j++)
                printf("%d_", temp[i * c + j]);
            printf("\n");
        }
        free(temp);
    }
}

```

```

    }
    else
        MPI_Gather(local_mat, local_r * c, MPI_INT, NULL, local_r * c,
            MPI_INT, 0, MPI_COMM_WORLD);
}

void PrintVector(int *v, int n, int my_rank)
{
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
            printf("%d_", v[i]);
        printf("\n");
    }
}

void MatVecMult(int *mat, int *vec, int *res, int local_r, int c)
{
    for (int i = 0; i < local_r; i++)
    {
        res[i] = 0;
        for (int j = 0; j < local_r * c; j++)
            res[i] += mat[i * c + j] * vec[j];
    }
}

int main()
{
    int comm_sz;
    int my_rank;

    MPI_Init(NULL, NULL);

    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    int r, c;
    InputDim(&r, my_rank);
    InputDim(&c, my_rank);
    int *mat = calloc(r / comm_sz * c, sizeof(int));

```

```
int *vec = calloc(c, sizeof(int));
InputVector(vec, c, my_rank);
InputMatrix(r, c, mat, my_rank, r / comm_sz);
PrintVector(vec, c, my_rank);
PrintDistributedMatrix(r, c, mat, my_rank, r / comm_sz);
int* res = calloc(r, sizeof(int));
MatVecMult(mat, vec, res, r / comm_sz, c);
PrintDistributedVector(res, r, my_rank, comm_sz);

MPI_Finalize();
}
```

2.7 MPI: Scatterv, Gatherv

Решить задачи предыдущего семинара без предположения, что размер вектора или матрицы делится нацело на количество процессов.

```
int MPI_Scatterv(
    const void *sendbuf,
    const int sendcounts[],
    const int displs[],
    MPI_Datatype sendtype,
    void *recvbuf,
    int recvcount,
    MPI_Datatype recvtype,
    int root,
    MPI_Comm comm
);
```

Принципиальными являются второй и третий аргументы. Оба массива имеют длину, равную количеству процессов. Элемент с индексом i массива `sendcounts` содержит длину подмассива `sendbuf`, которая достается процессу i . Элемент с индексом i массива `displs` содержит начальный индекс `sendbuf`, с которого отсчитывается `sendcounts[i]` элементов.

```
int MPI_Gatherv(
    const void *sendbuf,
    int sendcount,
    MPI_Datatype sendtype,
    void *recvbuf,
    const int recvcounts[],
    const int displs[],
    MPI_Datatype recvtype,
    int root,
    MPI_Comm comm
);
```

Задача 1. Реализовать скалярное произведение векторов.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

void InputDim(int *n, int my_rank)
{
```

```

    if (my_rank == 0)
    {
        scanf("%d", n);
    }
    MPI_Bcast(n, 1, MPI_INT, 0, MPI_COMM_WORLD);
}

void InputVector(int *v, int n, int my_rank,
    int *sizes, int *displs)
{
    if (my_rank == 0)
    {
        int *temp = calloc(n, sizeof(int));

        for (int i = 0; i < n; i++)
        {
            temp[i] = rand() % 10;
        }
        MPI_Scatterv(temp, sizes, displs, MPI_INT, v,
            sizes[my_rank], MPI_INT, 0, MPI_COMM_WORLD);
        free(temp);
    }
    else
        MPI_Scatterv(NULL, sizes, displs, MPI_INT, v,
            sizes[my_rank], MPI_INT, 0, MPI_COMM_WORLD);
}

void PrintVector(int *v, int n, int my_rank,
    int *sizes, int *displs)
{
    int *temp = calloc(n, sizeof(int));
    MPI_Gatherv(v, sizes[my_rank], MPI_INT, temp, sizes, displs,
        MPI_INT, 0, MPI_COMM_WORLD);
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
            printf("%d_", temp[i]);
        printf("\n");
    }
    free(temp);
}

```

```
void DotProd(int *v1, int *v2, int local_n, int my_rank)
{
    int local_res = 0, res = 0;
    for (int i = 0; i < local_n; i++)
        local_res += v1[i] * v2[i];
    MPI_Reduce(&local_res, &res, 1, MPI_INT,
    MPI_SUM, 0, MPI_COMM_WORLD);
    if (my_rank == 0)
        printf("%d\n", res);
}

void BuildSize(int n, int comm_sz, int *sizes)
{
    for (int i = 0; i < comm_sz; i++)
    {
        sizes[i] = n / comm_sz;
        if (i < n % comm_sz)
            sizes[i]++;
    }
}

void BuildDisplacements(int comm_sz, int *displs, int *sizes)
{
    displs[0] = 0;
    for (int i = 1; i < comm_sz; i++)
    {
        displs[i] = displs[i - 1] + sizes[i - 1];
    }
}

int main()
{
    int comm_sz;
    int my_rank;

    MPI_Init(NULL, NULL);

    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
```

```

int n;
InputDim(&n, my_rank);

int *sizes = calloc(comm_sz, sizeof(int));
int *displacements = calloc(comm_sz, sizeof(int));

BuildSize(n, comm_sz, sizes);
BuildDisplacements(comm_sz, displacements, sizes);

int *vec1, *vec2;
vec1 = calloc(sizes[my_rank], sizeof(int));
vec2 = calloc(sizes[my_rank], sizeof(int));

InputVector(vec1, n, my_rank, sizes, displacements);
InputVector(vec2, n, my_rank, sizes, displacements);
PrintVector(vec1, n, my_rank, sizes, displacements);
PrintVector(vec2, n, my_rank, sizes, displacements);
DotProd(vec1, vec2, sizes[my_rank], my_rank);

MPI_Finalize();
}

```

Задача 2. Умножить матрицу на вектор, используя построчное разбиение матрицы.

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

void InputDim(int *n, int my_rank)
{
    if (my_rank == 0)
    {
        scanf("%d", n);
    }
    MPI_Bcast(n, 1, MPI_INT, 0, MPI_COMM_WORLD);
}

void InputVector(int *v, int n, int my_rank)
{
    if (my_rank == 0)

```

```

    {
        for (int i = 0; i < n; i++)
        {
            v[i] = rand() % 10;
        }
    }
    MPI_Bcast(v, n, MPI_INT, 0, MPI_COMM_WORLD);
}

void PrintDistributedVector(int *v, int n, int my_rank,
    int *sizes, int *displs)
{
    int *temp = calloc(n, sizeof(int));
    MPI_Gatherv(v, sizes[my_rank], MPI_INT, temp, sizes, displs,
        MPI_INT, 0, MPI_COMM_WORLD);
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
            printf("%d_", temp[i]);
        printf("\n");
    }
    free(temp);
}

void InputMatrix(int r, int c, int *mat, int my_rank,
    int *sizes, int *displs)
{
    if (my_rank == 0)
    {
        int *temp = calloc(r * c, sizeof(int));
        if (my_rank == 0)
            for (int i = 0; i < r * c; i++)
                temp[i] = rand() % 10;
        MPI_Scatterv(temp, sizes, displs, MPI_INT,
            mat, sizes[my_rank], MPI_INT, 0, MPI_COMM_WORLD);
        free(temp);
    }
    else
    {
        MPI_Scatterv(NULL, sizes, displs, MPI_INT,
            mat, sizes[my_rank], MPI_INT, 0, MPI_COMM_WORLD);
    }
}

```

```

    }
}

void PrintDistributedMatrix(int r, int c, int *local_mat,
    int my_rank, int *sizes, int *displs)
{
    if (my_rank == 0)
    {
        int *temp = calloc(r * c, sizeof(int));
        MPI_Gatherv(local_mat, sizes[my_rank], MPI_INT, temp, sizes,
            displs, MPI_INT, 0, MPI_COMM_WORLD);
        for (int i = 0; i < r; i++)
        {
            for (int j = 0; j < c; j++)
                printf("%d_", temp[i * c + j]);
            printf("\n");
        }
        free(temp);
    }
    else
        MPI_Gatherv(local_mat, sizes[my_rank], MPI_INT, NULL, sizes,
            displs, MPI_INT, 0, MPI_COMM_WORLD);
}

void PrintVector(int *v, int n, int my_rank)
{
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
            printf("%d_", v[i]);
        printf("\n");
    }
}

void MatVecMult(int *mat, int *vec, int *res, int local_r, int c)
{
    for (int i = 0; i < local_r; i++)
    {
        res[i] = 0;
        for (int j = 0; j < local_r * c; j++)
            res[i] += mat[i * c + j] * vec[j];
    }
}

```

```

    }
}

void BuildSize(int r, int c, int comm_sz, int *sizes)
{
    for (int i = 0; i < comm_sz; i++)
    {
        sizes[i] = r / comm_sz;
        if (i < r % comm_sz)
            sizes[i]++;
        sizes[i] *= c;
    }
}

void BuildDisplacements(int comm_sz, int *displs, int *sizes)
{
    displs[0] = 0;
    for (int i = 1; i < comm_sz; i++)
    {
        displs[i] = displs[i - 1] + sizes[i - 1];
    }
}

int main()
{
    int comm_sz;
    int my_rank;

    MPI_Init(NULL, NULL);

    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    int r, c;
    InputDim(&r, my_rank);
    InputDim(&c, my_rank);
    int* sizes_mat = calloc(comm_sz, sizeof(int));
    int* displacements_mat = calloc(comm_sz, sizeof(int));
    int* sizes_vec = calloc(comm_sz, sizeof(int));
    int* displacements_vec = calloc(comm_sz, sizeof(int));

```

```
BuildSize(r, c, comm_sz, sizes_mat);
BuildDisplacements(comm_sz, displacements_mat, sizes_mat);

BuildSize(r, 1, comm_sz, sizes_vec);
BuildDisplacements(comm_sz, displacements_vec, sizes_vec);

int *mat = calloc(sizes_mat[my_rank], sizeof(int));
int *vec = calloc(c, sizeof(int));
InputVector(vec, c, my_rank);
InputMatrix(r, c, mat, my_rank, sizes_mat, displacements_mat);
PrintVector(vec, c, my_rank);
PrintDistributedMatrix(r, c, mat, my_rank,
    sizes_mat, displacements_mat);
int *res = calloc(r, sizeof(int));
MatVecMult(mat, vec, res, r / comm_sz, c);
PrintDistributedVector(res, r, my_rank,
    sizes_vec, displacements_vec);

MPI_Finalize();
}
```

2.8 MPI: производные типы данных

Создание нового типа данных, представляющего собой `count` идущих подряд элементов типа `oldtype`:

```
int MPI_Type_contiguous(
    int count,           /* in */
    MPI_Datatype oldtype, /* in */
    MPI_Datatype* newtype /* out */
);
```

Создание блочно-циклического разбиения вектора:

```
int MPI_Type_vector(
    int count,
    int blocklength,
    int stride,
    MPI_Datatype oldtype,
    MPI_Datatype *newtype
);
```

Создание произвольного набора компонент массива:

```
int MPI_Type_indexed(
    int count,
    const int *array_of_blocklengths,
    const int *array_of_displacements,
    MPI_Datatype oldtype,
    MPI_Datatype *newtype
);
```

Отметим, что `MPI_Type_vector` нельзя использовать с функциями `MPI_Gather` и `MPI_Scatter`. Поэтому в листинге 2.19 эти коллективные коммуникации заменяются на циклы с точечными коммуникациями.

Листинг 2.18: Демонстрация использования `MPI_Type_contiguous`

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
```

```
MPI_Datatype type_contiguous;
```

```
void InputDim(int *n, int my_rank)
{
    if (my_rank == 0)
```

```
{
    scanf("%d", n);
}
MPI_Bcast(n, 1, MPI_INT, 0, MPI_COMM_WORLD);
}

void InputVector(int *v, int n, int my_rank, int comm_sz)
{
    int *temp = calloc(n, sizeof(int));
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
        {
            temp[i] = i;
        }
        MPI_Scatter(temp, 1, type_contiguous, v, 1,
            type_contiguous, 0, MPI_COMM_WORLD);
    }
    else
        MPI_Scatter(temp, 1, type_contiguous, v, 1,
            type_contiguous, 0, MPI_COMM_WORLD);

    free(temp);
}

void PrintDistributedVector(int *v, int n, int my_rank, int comm_sz)
{
    int *temp = calloc(n, sizeof(int));
    MPI_Gather(v, 1, type_contiguous, temp, 1,
        type_contiguous, 0, MPI_COMM_WORLD);
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
            printf("%d_", temp[i]);
        printf("\n");
    }
    free(temp);
}

int main()
{
```

```

int comm_sz;
int my_rank;

MPI_Init(NULL, NULL);

MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

int n;
InputDim(&n, my_rank);

MPI_Type_contiguous(n / comm_sz, MPI_INT, &type_contiguous);
MPI_Type_commit(&type_contiguous);

int *vec;
vec = calloc(n / comm_sz, sizeof(int));
InputVector(vec, n, my_rank, comm_sz);
PrintDistributedVector(vec, n, my_rank, comm_sz);

MPI_Type_free(&type_contiguous);

free(vec);
MPI_Finalize();
}

```

Листинг 2.19: Блочнo-циклическое разбиение массива с использованием `MPI_Type_vector`

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

MPI_Datatype type_vector;
int block_len = 2;
int block_num = 3;

// 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
// block_len = 2, block_num = 3
// 0: 0 1 6 7 12 13
// 1: 2 3 8 9 14 15
// 2: 4 5 10 11 16 17

```

```

void InputDim(int *n, int my_rank)
{
    if (my_rank == 0)
    {
        scanf("%d", n);
    }
    MPI_Bcast(n, 1, MPI_INT, 0, MPI_COMM_WORLD);
}

void InputVector(int *v, int n, int my_rank, int comm_sz)
{
    int *temp = calloc(n, sizeof(int));
    if (my_rank == 0)
    {
        for (int i = 0; i < n; i++)
        {
            temp[i] = i;
        }
        int kostbl = 0;
        for (int i = 0; i < n; i += block_len*comm_sz) {
            for (int j = 0; j < block_len; ++j) {
                v[kostbl] = temp[i + j];
                ++kostbl;
            }
        }
        for (int i = 1; i < comm_sz; ++i) {
            MPI_Send(temp + i*block_len, 1, type_vector, i, 0, MPI_COMM_WORLD);
        }
    }
    else {
        MPI_Recv(v, n / comm_sz, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    free(temp);
}

void PrintDistributedVector(int *v, int n,
    int my_rank, int comm_sz)
{
    int *temp = calloc(n, sizeof(int));

```

```

    if (my_rank == 0)
    {
        for (int i = 1; i < comm_sz; ++i) {
            MPI_Recv(temp + i*block_len, 1, type_vector,
                `i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        }
        int kostbl = 0;
        for (int i = 0; i < n; i += block_len*comm_sz) {
            for (int j = 0; j < block_len; ++j) {
                temp[i + j] = v[kostbl];
                ++kostbl;
            }
        }
        for (int i = 0; i < n; i++)
            printf("%d_", temp[i]);
        printf("\n");
    } else {
        MPI_Send(v, n/comm_sz, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }
    free(temp);
}

int main()
{

    int comm_sz;
    int my_rank;

    MPI_Init(NULL, NULL);

    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    int n;
    InputDim(&n, my_rank);

    MPI_Type_vector(block_num, block_len, comm_sz*block_len,
        MPI_INT, &type_vector);
    MPI_Type_commit(&type_vector);

```

```

    int *vec;
    vec = calloc(n / comm_sz, sizeof(int));
    InputVector(vec, n, my_rank, comm_sz);
    PrintDistributedVector(vec, n, my_rank, comm_sz);

    MPI_Type_free(&type_vector);

    free(vec);
    MPI_Finalize();
}

```

Листинг 2.20: Выделение правой верхней подматрицы (демонстрация MPI_Type_indexed)

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main()
{

    int comm_sz;
    int my_rank;

    MPI_Init(NULL, NULL);

    comm_sz = 2;
    //MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Datatype type_indexed;
    int array_of_blocklengths[4] = {4, 3, 2, 1};
    int array_of_displacements[4] = {0, 5, 10, 15};

    MPI_Type_indexed(4, array_of_blocklengths, array_of_displacements,
        MPI_INT, &type_indexed);
    MPI_Type_commit(&type_indexed);
    int mat[16], mat1[10];
    if (my_rank == 0)
    {
        for (int i = 0; i < 16; ++i)
        {
            mat[i] = i;

```

```
    }  
    MPI_Send(mat, 1, type_indexed, 1, 0, MPI_COMM_WORLD);  
  
} else {  
    MPI_Recv(mat1, 10, MPI_INT, 0, 0,  
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
    for (int i=0; i<10; ++i) {  
        printf("%3d", mat1[i]);  
    }  
    printf("\n");  
}  
  
MPI_Type_free(&type_indexed);  
MPI_Finalize();  
}
```

2.9 OpenMP: потоки, таскинг

2.10 OpenMP: очередь

2.11 OpenMP: вычислительная геометрия

2.12 CUDA

2.13 **CUDA**

Глава 3

Лабораторные работы

3.1 Pthreads

!!!Дедлайн 22 октября 23:59 МСК!!!

3.1.1 Вычисление числа π

Описание задачи. Требуется определить число π по методу Монте-Карло.

Именно, пусть в квадрат со стороной $2r$ бросается дротик (монетка, кошка или другой предмет). Нарисуем внутри квадрата вписанную окружность с радиусом r . Ясно, что отношение количества попаданий в окружность к общему количеству бросков будет равно $\frac{\pi}{4}$. Следовательно, число π можно представить как

$$\pi \approx 4 \frac{\text{Количество попаданий в окружность}}{\text{Общее количество бросков}} \quad (3.1)$$

Формат входных данных. Запуск программы из командной строки `./program nthreads ntrials`, где `nthreads` — количество потоков, `ntrials` — общее количество попыток.

Формат выходных данных. Вывести на экран получившееся значение числа π .

3.1.2 Множество Мандельброта

Описание задачи. Множество Мандельброта — это совокупность всех чисел $c \in \mathbb{C}$, для которых последовательность

$$z_{n+1} = z_n^2 + c \quad (3.2)$$

является ограниченной для всех значений $n \in \mathbb{N}$. Иными словами, число c принадлежит множеству Мандельброта тогда и только тогда, когда

$$(\exists A \in \mathbb{R}) \quad (\forall n \in \mathbb{N}) \quad |z_n| < A$$

Можно доказать, что в последнем равенстве A можно взять равным двум, т.е. число c принадлежит множеству Мандельброта тогда и только тогда, когда

$$(\forall n \in \mathbb{N}) \quad |z_n| < 2, \quad z_1 = 0$$

Формат входных данных. Запуск из командной строки `./program nthreads npoints`,

где `nthreads` — количество потоков,
`npoints` — количество точек.

Формат выходных данных. Файл формата `csv` с координатами точек множества Мандельброта.

3.1.3 Реализация read-write lock

Описание задачи.

В `pthread` есть тип данных `pthread_rwlock_t`. Инициализация и уничтожение осуществляются по знакомой схеме:

```
int pthread_rwlock_init(
    pthread_rwlock_t*    lock,    /*out*/
    pthread_rwlockattr_t attr    /*in*/
);
```

```
int pthread_rwlock_destroy(
    pthread_rwlock_t*    lock    /*out*/
);
```

Данный тип похож на мьютексы, но допускает две функции `lock`:

```
int pthread_rwlock_rdlock(
    pthread_rwlock_t*    lock    /*in/out*/
);
```

```
int pthread_rwlock_wrlock(
    pthread_rwlock_t*    lock    /*in/out*/
);
```

Как и для мьютексов, есть лишь одна функция типа `unlock`:

```
int pthread_rwlock_unlock(
    pthread_rwlock_t*    lock    /*in/out*/
);
```

Когда поток получает блокировку чтения, то сколько угодно других потоков могут также получить блокировку на чтение, но ни один поток не получит блокировку на запись, пока все читающие потоки не вызовут функцию `unlock` для данной переменной.

Когда поток получает блокировку на запись, то ни один другой поток не сможет получить блокировку ни на чтение, ни на запись.

Как это можно организовать?.

Создать структуру, включающую в себя:

- Мьютекс
- Две условные переменные (одну для читателей, другую для писателей)
- Счетчик читателей (сколько потоков в данный момент читают)
- Счетчик количества потоков, ожидающих получения блокировки на чтение
- Счетчик количества потоков, ожидающих получения блокировки на запись
- Флаг, показывающий, получил ли блокировку хотя бы один писатель в данный момент

Что нужно сделать

1. Создать свою реализацию типа `rwlock` и функций `rdlock`, `wrlock`.
2. Апробировать эту реализацию на примере односвязного списка. См. файл `pth_ll_rwl.c`. Сравнить производительность своей реализации с библиотечной.

Обратите внимание, что в программе `pth_ll_rwl` используется свой генератор случайных чисел, т.к. стандартный `rand()` не является потокобезопасным: `seed` там один на все потоки.

3.1.4 Критерии оценивания

На оценку 5 достаточно выполнить первую или вторую задачу.

На оценку 7 достаточно выполнить первую и вторую задачи.

На оценку 10 нужно выполнить все три задачи.

Необходимо предоставить исходные коды программ и текст отчета, который бы содержал описание алгоритма и оценки времени работы, ускорения и эффективности разработанных программ в зависимости от объема входных данных и количества потоков. Провести анализ полученных результатов, сделать выводы.

За что может быть снижена оценка:

- Неоптимальность или ошибочность алгоритма.
- Неполные или недостоверные данные в отчете, отсутствие анализа результатов.

3.2 MPI

3.2.1 Задания

1. Реализовать алгоритмы для умножения матрицы на вектор, используя разбиение по строкам, по столбцам и по блокам. Проведите численные эксперименты, измеряя время выполнения функций при различных размерах входных данных. Постройте графики времени выполнения, ускорения и эффективности. Сделайте выводы о быстродействии рассмотренных трех алгоритмов.
2. Реализовать алгоритм матричного умножения с разбиением на блоки по алгоритму Кэннона. Выполнить замеры времени работы, определить ускорение в зависимости от размерности задачи и количества процессов.
3. Реализовать решение задачи Дирихле для определения стационарного распределения температур на пластине с внешними источниками тепла, на краях которой поддерживается заданный температурный режим:

$$\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{\partial^2 u(x, y)}{\partial y^2} = f(x, y), \quad 0 < x < 1, \quad 0 < y < 1 \quad (3.3)$$

$$u(0, y) = u(1, y) = u(x, 0) = u(x, 1) = c$$

При организации параллельных вычислений использовать метод Гаусса-Зейделя при волновой схеме организации вычислений и блочном представлении обрабатываемых данных.

Выполнить замеры времени работы, определить ускорение в зависимости от размерности задачи, количества процессов и размера блоков.

3.2.2 Критерии оценивания

Выполнять в группах численностью 1, 2 или 3 человека.

Для получения оценки 4 достаточно реализовать один алгоритм из 1 задания (но не строчное разбиение!). Выполнить замеры ускорения и эффективности.

Плюс 2 балла за выполнение первого задания.

Плюс 2 балла за второе задание.

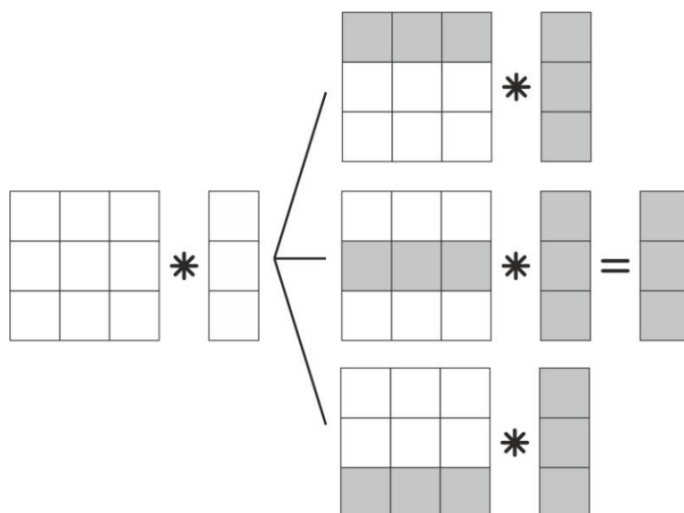


Рис. 3.1: Схематическое представление умножения матрицы на вектор при разбиении по строкам

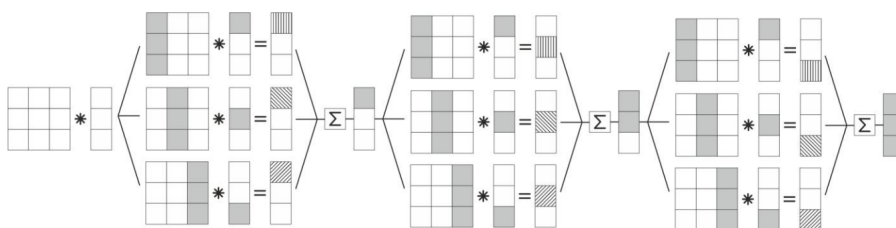


Рис. 3.2: Схематическое представление алгоритма умножения матрицы на вектор при разбиении по столбцам

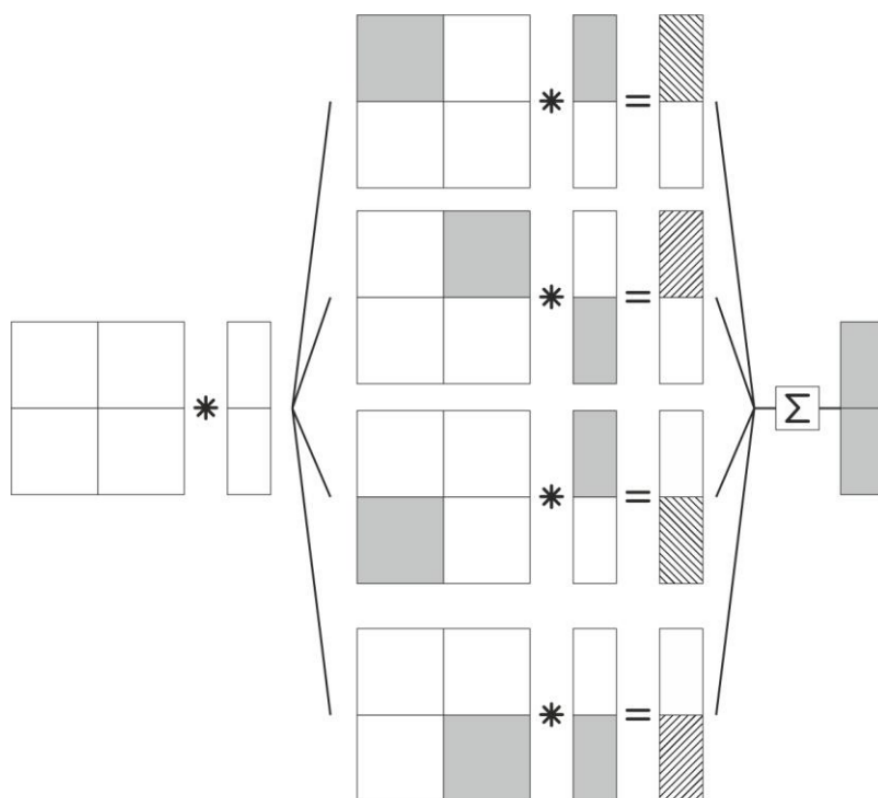


Рис. 3.3: Схематическая иллюстрация умножения матрицы на вектор при разбиении на блоки

Плюс 2 балла за третье задание.

Дедлайн — 23:59 3 декабря 2024 года.